# CodeSimilar

## TECHNICAL MANUALS

*Academic Integrity Code Submission Checker*

Installation Manual | User Manual | Source Code Documentation

Brock University | 2026

COSC 4P02

Group 8

# Table of Contents


- **Installation Manual ..... 8**

**1. Introduction ..... 9**

1.1 Purpose ..... 9
1.2 Scope ..... 9
1.3 Intended Audience ..... 9
1.4 Prerequisites ..... 9
1.5 How to use this document ..... 9

**2. System Overview ..... 10**

2.1 System Architecture ..... 10
*2.1.1 Frontend (User Interface) ..... 10*
*2.1.2 Backend API ..... 10*
*2.1.3 Similarity Analysis Engine ..... 10*
*2.1.4 Database (Relational Database) ..... 11*
*2.1.5 File Storage System ..... 11*
2.2 System Interaction Flow ..... 11
2.3 Deployment Model ..... 12
..... 12

**3. System Requirements ..... 13**

3.1 Hardware Requirements ..... 13
3.2 Software Requirements ..... 13
3.3 Environment Configuration ..... 13
3.4 Database Setup ..... 14
3.5 Ports and Accesses ..... 14

**4. Installation Methods ..... 15**

4.1 Hosted Deployment ..... 15
4.2 Local Deployment ..... 15
4.3 Verify Installation ..... 16

**5. Security and Privacy Considerations ..... 17**

**6. Basic Installation Issues ..... 18**

- **User Manual ..... 19**

**1. Introduction ..... 20**

1.1 Purpose of This Manual ..... 20
1.2 Intended Audience ..... 20
1.3 What is CodeSimilar? ..... 20
1.4 How to Use This Manual ..... 20
1.5 Getting Help ..... 21

**2. System Overview ..... 22**

2.1 Core Workflow ..... 22

2.2 Key Features ..... 22
2.3 Supported File Types ..... 22
2.4 Deployment Options ..... 23
**3. Getting Started ..... 24**
3.1 Accessing the Hosted Platform ..... 24
3.2 Running the Platform Locally ..... 24
*3.2.1 Prerequisites ..... 24*
*3.2.2 Starting the Local Stack ..... 24*
3.3 Instructor Account Setup ..... 25
*3.3.1 Signing In with Google ..... 25*
*3.3.2 Creating a Manual Account ..... 25*
3.4 Browser & System Requirements ..... 26
**4. Instructor Guide ..... 27**
4.1 Dashboard Walkthrough ..... 27
4.2 Creating a New Assignment ..... 28
4.3 Uploading a Repository ..... 28
4.4 Managing Student Submissions ..... 29
4.5 Archive Policy ..... 30
**5. Running Similarity Analysis ..... 31**
5.1 Internal Analysis ..... 31
5.2 Cross-Repository Analysis ..... 31
5.3 Understanding Similarity Scores ..... 32
**6. Reviewing Results ..... 33**
6.1 Pairwise Similarities Panel ..... 33
6.2 Side-by-Side Code Comparison ..... 33
6.3 Analytics Overview & Visualisations ..... 34
*6.3.1 Analytics Overview — Bar Graph ..... 34*
*6.3.2 Similarity Heatmap ..... 35*
*6.3.3 Collusion Cluster Graph ..... 35*
6.4 Exporting Submissions ..... 36
6.5 Downloading the PDF Report ..... 37
**7. Student Submission Guide ..... 38**
7.1 What You Need Before Submitting ..... 38
7.2 Submitting Your Work ..... 38
7.3 File Preparation Guidelines ..... 39
7.4 After Submission ..... 39
**8. Frequently Asked Questions ..... 40**
8.1 For Instructors & TAs ..... 40
*Can I run the analysis more than once for the same assignment? ..... 40*
*Can I compare submissions against multiple repositories at the same time? ..... 40*
*What happens to an assignment after the due date? ..... 40*
*Will archiving an assignment delete the student submissions? ..... 40*
*Why are some similarity scores higher than expected for clearly original work? ..... 40*

*Can I use CodeSimilar on a mobile device?* ..... *40*
8.2 For Students ..... 41
*Where do I get my access key?* ..... *41*
*Can I submit more than once?* ..... *41*
*Will the instructor see my name next to my similarity score?* ..... *41*
*What file formats are accepted?* ..... *41*
*I submitted the wrong file. What should I do?* ..... *41*
**9. Support & Contact ..... 42**
9.1 Contacting the Development Team ..... 42
9.2 Known Limitations ..... 42
9.3 Reporting a Bug ..... 42
**10. Glossary ..... 43**
- **Source Code Documentation ..... 44**

**1. System Overview ..... 49**
Technology Stack ..... 49
**2. Backend : Core Infrastructure ..... 49**
config.py ..... 49
*Settingsclass* ..... *49*
*get_settings() → Settings* ..... *50*
database.py ..... 50
*get_db() → Generator[Session, None, None]* ..... *50*
security.py ..... 50
**3. Backend : Analysis Engine ..... 50**
tokenizer.py ..... 50
fingerprint.py ..... 51
scorer.py ..... 51
*calculate_similarity Return Value* ..... *52*
**4. Backend : API Layer ..... 52**
dependencies.py ..... 52
*get_current_user(...) → Instructor* ..... *52*
analysis.py ..... 52
auth.py ..... 53
student.py ..... 53
repository.py ..... 54
health.py ..... 54
*health_check() - GET /api/v1/health* ..... *54*
**5. Backend : ORM Models ..... 54**
**6. Backend : Repositories (Data Access Layer) ..... 55**
base_repository.py - BaseRepository (generic class) ..... 55
Domain Repositories ..... 55
**7. Backend : Schemas (Validation Layer) ..... 55**

Schema Pattern (all domains) ..... 55
Notable Schemas ..... 56
**8. Backend : Services (Business Logic) ..... 56**
analysis_service.py : AnalysisService ..... 56
archival_service.py ..... 56
encryption_service.py : EncryptionService ..... 56
sanitization_service.py : SanitizationService ..... 57
storage_service.py : StorageService ..... 57
**9. Backend : Application Entry Point ..... 57**
**10. Frontend : Application Root ..... 58**
frontend/src/main.tsx ..... 58
frontend/src/App.tsx ..... 58
**11. Frontend : Pages ..... 58**
InstructorLogin.tsx ..... 58
InstructorRegister.tsx ..... 58
InstructorProfile.tsx ..... 58
Dashboard.tsx ..... 59
AssignmentDetails.tsx ..... 59
ComparisonView.tsx ..... 59
Repositories.tsx ..... 59
StudentPortal.tsx ..... 60
Other Pages ..... 60
**12. Frontend : Assignment Details Components ..... 60**
assignmentDetailsUtils.ts ..... 60
SimilarityMatrixTab.tsx ..... 60
Other Assignment Detail Components ..... 61
**13. Frontend : Comparison View Components ..... 61**
comparisonUtils.ts ..... 61
useComparisonData.ts ..... 61
Other Comparison Components ..... 62
**14. Frontend : Dashboard Components ..... 62**
**15. Frontend : Student Portal Components ..... 63**
studentPortalUtils.ts ..... 63
submissionReceiptPdf.ts ..... 63
Other Student Portal Components ..... 63
**16. Frontend : UI Primitives ..... 64**
**17. Frontend : Custom Hooks ..... 64**
useAssignmentDetailsData.ts ..... 64
useDashboardData.ts ..... 64
useFileDrop.ts ..... 64

useStudentPortalSubmission.ts ..... 64

**18. Frontend : Libraries & Utilities ..... 65**

apiClient.ts ..... 65
generateComparisonPDF.ts ..... 65
generatePDFReport.ts ..... 65

**19. Frontend : Type Declarations ..... 66**

auth.ts ..... 66
dashboard.ts ..... 66
jszip.d.ts ..... 66

**20. Frontend : API Client Modules ..... 66**

authApi.ts ..... 66
dashboardApi.ts ..... 66

**21. Module Dependency Map ..... 67**

Backend ..... 67
Frontend ..... 67

# CodeSimilar

# INSTALLATION MANUAL

## Installation Manual

Comprehensive Installation & Site Preparation Guide

Brock University | 2026

COSC 4P02

Group 8

# 1. Introduction

## 1.1 Purpose

This Installation Manual provides detailed instructions for installing and running the CodeSimilar Academic Integrity Code Comparison System. It includes system architecture, setup requirements, and step-by-step deployment instructions for both local and hosted environments. The system is designed as a Docker-based web application, allowing easy deployment across different platforms.

## 1.2 Scope

This document covers:

1. System architecture overview
2. Hardware and software requirements
3. Installation procedures (local and hosted)

## 1.3 Intended Audience

This manual is intended for:

1. System administrators
2. Instructors deploying the system
3. Technical evaluators (e.g., TAs, professors)

## 1.4 Prerequisites

Maintenance staff should have:
Basic familiarity with Docker and Docker Compose CLI commands
SSH or terminal access to the host server
Read access to environment configuration files (.env)
Access to project repository (GitHub)

## 1.5 How to use this document

This document is intended to guide users through the installation, setup, and basic operation of the CodeSimilar system. It should be used as a reference during both initial deployment and future system maintenance.

1. Before installation, users should review Section 3 (System Requirements) to ensure their environment meets all necessary prerequisites.
2. During setup, users should follow Section 4 (Installation Methods) for either hosted or local deployment.
3. After installation, users should refer to Section 4.3 (Verify Installation) to confirm that the system is functioning correctly.
4. For common setup-related issues, users may consult Section 5 (Basic Installation Issues).

This document is structured sequentially, so users are encouraged to follow the sections in order during first-time setup, and revisit specific sections as needed for support.

# 2. System Overview

This chapter describes the overall architecture and components of the CodeSimilar system.

---

## 2.1 System Architecture

The CodeSimilar system is designed using a Dockerized microservices architecture, where each major component runs in an isolated container. This approach improves scalability, maintainability, and portability, allowing the system to be deployed on any environment that supports Docker.
The architecture separates the system into distinct layers, each responsible for a specific function

### 2.1.1 Frontend (User Interface)

The frontend is a web-based interface accessible through modern browsers. It provides:

- A student submission portal for uploading assignments using an access key
- An instructor dashboard for managing assignments, running analysis, and viewing results

The frontend communicates with the backend through secure HTTP (HTTPS) requests.

### 2.1.2 Backend API

The backend acts as the central controller of the system. It is responsible for:

1. Handling user requests (submission, login, analysis)
2. Managing assignment configurations and repositories
3. Coordinating communication between all system components

It also enforces validation rules such as access key verification and submission deadlines.

### 2.1.3 Similarity Analysis Engine

This component performs the core functionality of the system. It:

1. Processes sanitized source code
2. Converts code into tokens (lexical analysis)
3. Generates fingerprints using algorithms such as winnowing
4. Computes similarity scores between submissions

The engine operates independently to allow efficient batch processing of multiple files.

### 2.1.4 Database (Relational Database)

A relational database (e.g., PostgreSQL) is used to store:

1. Assignment details
2. Submission metadata (encrypted student information)
3. Similarity results and analysis data

The database ensures data integrity and supports efficient querying for analysis and reporting.

### 2.1.5 File Storage System

All submitted code files are stored in a file storage system after being anonymized. Key features include:

1. Replacement of original filenames with UUIDs
2. Storage of sanitized code (comments removed)
3. Retention of files for future comparisons

This separation of file storage from metadata enhances security and performance.

## 2.2 System Interaction Flow

The CodeSimilar system operates through a coordinated interaction between multiple components in the Dockerized architecture. The workflow for a typical submission and analysis process is as follows:

1. Submission Phase:
    1. A student accesses the frontend via a web browser and submits an assignment using an access key.
    2. The frontend sends the submission request to the backend API over HTTPS.
2. Processing & Sanitization:
    1. The backend validates the access key and submission constraints (e.g., deadline, submission limit).

The uploaded file is passed through the sanitization pipeline:

1. Comments and unnecessary metadata are removed.
2. The original filename is discarded and replaced with a UUID.
3. Student metadata (Name/ID) is separated and encrypted before storage.

3. Storage:
    2. The sanitized code file is stored in the file storage system.
    3. Encrypted metadata and submission details are stored in the relational database.
4. Analysis Phase:
    4. When the instructor initiates analysis, the backend triggers the similarity analysis engine.
    5. The engine retrieves relevant files from storage.
    6. Code is tokenized and transformed into fingerprints using algorithms such as winnowing.
    7. Pairwise comparisons are performed to compute similarity scores.
5. Results & Visualization:
    1. The analysis results are sent back to the backend.

2. The backend stores the results and forwards them to the frontend.
3. The instructor views results through dashboards, graphs, and comparison tools.

   This interaction ensures a modular, scalable, and secure flow of data across the system.

## 2.3 Deployment Model

All components are containerized using Docker and orchestrated using Docker Compose. This ensures:

1. Easy setup with a single command
2. Consistent runtime environments
3. Simplified deployment across different systems

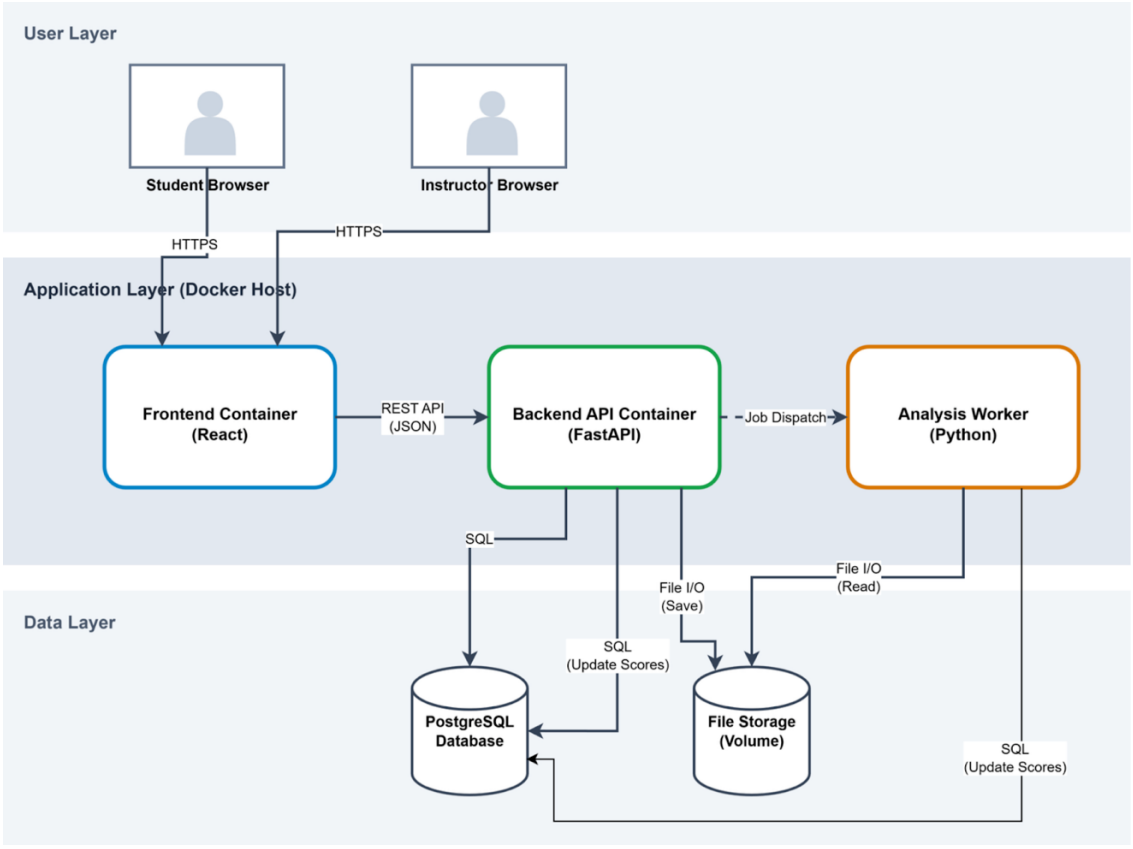



*Figure 1. Architecture Model:* Illustrates the Docker-based microservices architecture of the CodeSimilar system, showing the interaction between the frontend, backend, analysis engine, and data storage components.

# 3. System Requirements

## 3.1 Hardware Requirements

4. Minimum 8 GB RAM
5. 10 GB free disk space
6. Modern CPU

## 3.2 Software Requirements

7. Docker Desktop (latest version)
8. Web browser (Chrome, Edge, Firefox, Safari)
1. Internet connection (for hosted mode)

## 3.3 Environment Configuration

The system uses environment variables to configure core services such as the database, authentication, storage, and frontend integration. These variables are defined in a .env file located in the root directory of the project.
Before deployment, create a .env file by copying the provided template and updating all placeholder values.
The configuration includes the following key sections:
PostgreSQL Configuration:
Defines the database credentials and database name used by the system.

1. POSTGRES_USER – database username
2. POSTGRES_PASSWORD – secure password for database access
3. POSTGRES_DB – database name

Security Configuration:
Used for authentication and encryption.

1. SECRET_KEY – used for JWT token generation (must be at least 32 characters)
2. ENCRYPTION_KEY – used for encrypting sensitive fields such as student metadata
3. ENCRYPTION_LEGACY_KEYS – optional keys for backward compatibility during key rotation
4. ENCRYPTION_KEY_REQUIRED – ensures the system fails to start if encryption is not configured

Secure keys can be generated using:
openssl rand -hex 32
Object Storage (MinIO):
Used for storing anonymized code submissions.

1. MINIO_ACCESS_KEY – storage access username
2. MINIO_SECRET_KEY – storage access password

CORS Configuration:
Controls which domains are allowed to access the backend API.

1. ALLOWED_ORIGINS – set to the deployed frontend domain (no trailing slash)

Google OAuth Configuration (Optional):
Enables instructor login via Google Sign-In.

1. GOOGLE_CLIENT_ID
2. GOOGLE_CLIENT_SECRET
3. VITE_GOOGLE_CLIENT_ID

If these values are not provided, Google Sign-In will be disabled.
Frontend Configuration:

1. FRONTEND_URL – specifies the public URL of the frontend application

All placeholder values (e.g., CHANGE_ME) must be replaced with secure, environment-specific values before deployment. The .env file must not be committed to version control to prevent exposure of sensitive credentials

## 3.4 Database Setup

The system uses a relational database (e.g., PostgreSQL) to store metadata and analysis results.
Docker-Based Setup:
The database service is automatically created and initialized when running the system using Docker Compose.

```
docker compose up –build
```

## 3.5 Ports and Accesses

The application uses the following default ports:

2. Frontend:

http://localhost:3000

3. Backend API:

http://localhost:8080

4. Database:

localhost:5432

If these ports are already in use, they can be modified in the `.env` file or Docker configuration.

Ensure the correct URL is used when accessing the system after deployment.

# 4. Installation Methods

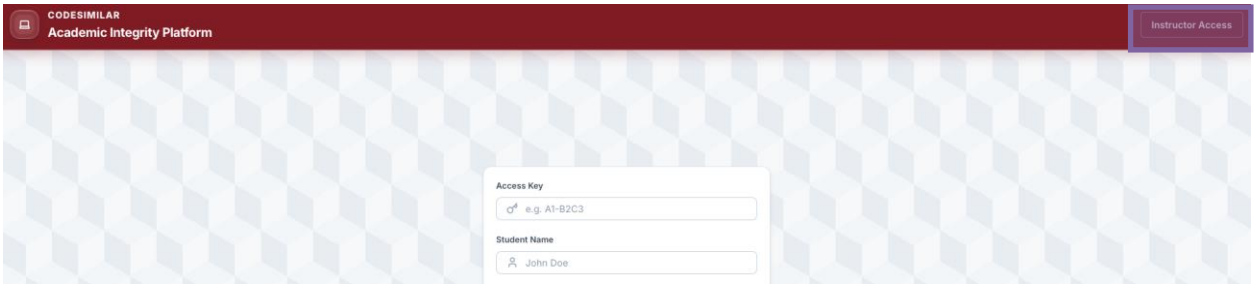
The system can be run in two ways:

---

## 4.1 Hosted Deployment

1. Open browser
2. Navigate to: https://codesimilar.com
3. Login as instructor:

Click on **Faculty Access** on the top right




Use Google Sign-In OR
Create an account manually

***NOTE:*** *No installation required.*

## 4.2 Local Deployment

**Step 1: Install Docker**

1. Download and install Docker Desktop
2. Ensure Docker is running

**Step 2: Open Project Source Code**

1. Extract/download project files
2. Navigate to project root folder

**Step 3: Run the Application**

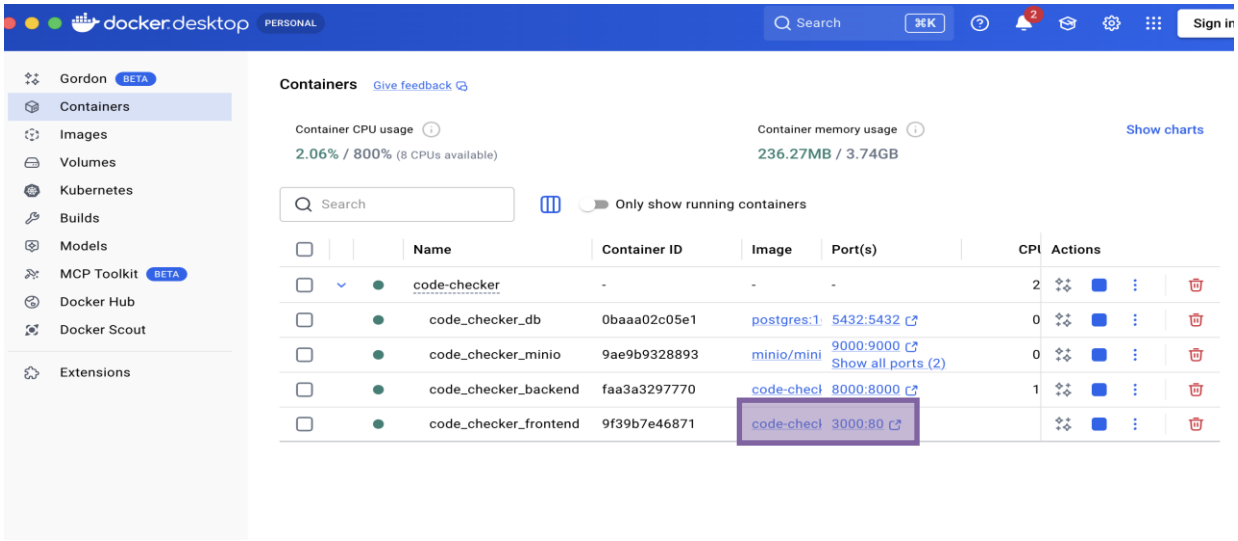
Open terminal and run:

```
docker compose up –build
```

***NOTE:*** *The docker desktop app should be opened before running the terminal*

### Step 4: Launch Application

- Go to Docker Desktop
- Click the running container link
- Access the system in browser




***NOTE:*** *Google Sign-In does NOT work locally*

5. Once logged in create account manually

In order to rebuild the containers, open terminal and run

```
docker compose down

docker compose up –build
```

## 4.3 Verify Installation

After completing local deployment, verify that the system is functioning correctly:

1. Open a web browser and navigate to: http://localhost:3000
2. Confirm that the homepage loads successfully.
3. Log in as an instructor or create a new account.
4. Create a test assignment and obtain an access key.
5. Submit a sample file through the student submission portal.
6. Run the analysis and confirm that similarity results are displayed.

Successful completion of these steps indicates that the system is correctly installed and operational.

*NOTE: Refer to User Manual on how to follow steps 3-6*

# 5. Security and Privacy Considerations

The CodeSimilar system is designed to comply with privacy regulations such as FIPPA and ensure secure handling of student data.

---

Key measures include:

1. Anonymization:

1. All submitted files are renamed using UUIDs.
2. Original filenames are discarded to prevent identity exposure.

2. Metadata Separation:

- Student Name and ID are stored separately from code files.
- Metadata is encrypted and only accessible to instructors.

3. Comment Removal:

1. All comments are stripped from code to remove potential identifying information.

4. Data Retention Policy:

1. Personal metadata is automatically deleted after a defined retention period of 30 days
2. Anonymized code is retained for future comparison.

5. No Student Accounts:

1. The system avoids persistent student accounts to minimize storage of personal data.

These measures ensure that the system maintains confidentiality, integrity, and compliance with institutional regulations.

# 6. Basic Installation Issues

| Issue | Description/Cause | Solution |
|---|---|---|
| Docker is not running | Docker service is not active on the system | Ensure Docker Desktop is installed and running before executing commands |
| Port already in use | Another application is using the required port | Modify port values in the .env file or stop the conflicting application |
| Application not accessible | Containers may not be running or incorrect URL is used | Run docker ps in the terminal to verify containers are running and ensure correct URL (e.g., http://localhost:3000) |

*NOTE: For advanced issue resolution and system maintenance procedures, refer to the Maintenance Manual.*

# CodeSimilar

# USER MANUAL

## User Manual

Interface Guide & User Support Documentation

Brock University | 2026

COSC 4P02

Group 8

# 1. Introduction

## 1.1 Purpose of This Manual

This User Manual provides comprehensive guidance for all users of the CodeSimilar Academic Integrity Platform. It covers the complete workflow from initial access and account setup through to similarity analysis, results interpretation, and report generation. The manual is intended to serve as the authoritative reference for both day-to-day operation and first-time onboarding.

## 1.2 Intended Audience

This manual addresses two distinct user groups:

| User Role | Description |
|---|---|
| Instructor / TA | Faculty members and teaching assistants responsible for creating assignments, uploading code repositories, running similarity analyses, and reviewing results for academic integrity purposes. |
| Student | Students enrolled in a course who are required to submit their code assignments through the CodeSimilar submission portal using an access key provided by their instructor. |

## 1.3 What is CodeSimilar?

CodeSimilar is a web-based academic integrity platform designed to assist instructors in detecting potential code plagiarism across student submissions. The system accepts source code in C, C++, Java, and related header formats, stores submissions securely, and applies a similarity analysis engine to identify structurally similar code pairs. Results are presented through an interactive instructor dashboard featuring side-by-side comparisons, similarity heatmaps, and collusion cluster visualisations.




## 1.4 How to Use This Manual

The manual is organized to mirror the natural workflow of the platform. Instructors should begin with Chapters 2 through 6; students may proceed directly to Chapter 7. A Frequently Asked

Questions section (Chapter 8) addresses the most common points of confusion, and a Glossary (Chapter 10) defines all technical terms used throughout.

## 1.5 Getting Help

Support for CodeSimilar is provided via email. If you encounter an issue not addressed in this manual, contact the development team at:

**pp21qh@brocku.ca**

Please include your name, role (instructor or student), a description of the issue, and any error messages observed.

# 2. System Overview

## 2.1 Core Workflow

The CodeSimilar workflow follows four sequential phases:

| Phase | Actor | Action |
|---|---|---|
| 1. Setup | Instructor | Create an assignment and distribute the generated access key to students. |
| 2. Submission | Student | Submit code files via the public submission portal using the access key. |
| 3. Analysis | Instructor | Run Internal Analysis or Cross-Repository Analysis against the collected submissions. |
| 4. Review | Instructor | Examine pairwise similarity scores, side-by-side code comparisons, and visual reports. |

## 2.2 Key Features

- Assignment and repository management with configurable due dates and archiving.
- Support for C, C++, Java source files and ZIP archives up to 50 MB.
- Internal similarity analysis across all submissions within an active assignment.
- Cross-repository analysis comparing active submissions against historical or reference repositories.
- Side-by-side highlighted code comparison with block-level match navigation.
- Visual analytics: bar graph, similarity heatmap, and collusion cluster graph.
- Downloadable PDF reports for academic integrity documentation.
- Student anonymization in all instructor-facing similarity reports.

## 2.3 Supported File Types

| File Type | Accepted Formats |
|---|---|
| Individual source files | .c \| .cpp \| .java \| .h |
| Compressed archives | .zip (containing any of the above) |
| Maximum file size | 50 MB per submission |

## 2.4 Deployment Options

CodeSimilar is available in two deployment modes. The appropriate mode depends on your access and environment:

| Mode | URL / Command | Notes |
|---|---|---|
| Hosted (recommended) | https://codesimilar.com | No local installation required. Google Sign-In available. Suitable for all users. |
| Local (development) | docker compose up --build | Requires Docker Desktop. Google Sign-In not available locally. Use test credentials. |

# 3. Getting Started

## 3.1 Accessing the Hosted Platform

The hosted deployment is the recommended access method for all instructors, TAs, and students. No installation is required.

1. Open a supported web browser (Chrome, Firefox, or Edge — latest version recommended).
2. Navigate to the platform: https://codesimilar.com
3. Instructors: click the Instructor Access button in the top-right corner of the page to proceed to the login screen.
4. Students: the submission form is displayed directly on the landing page.

## 3.2 Running the Platform Locally

Local execution is intended for development and evaluation purposes. It requires Docker Desktop to be installed and running on the host machine.

### 3.2.1 Prerequisites

- Docker Desktop installed and running (Windows, macOS, or Linux).
- The CodeSimilar source code repository cloned to your local machine.
- A terminal or command prompt with access to the project root directory.

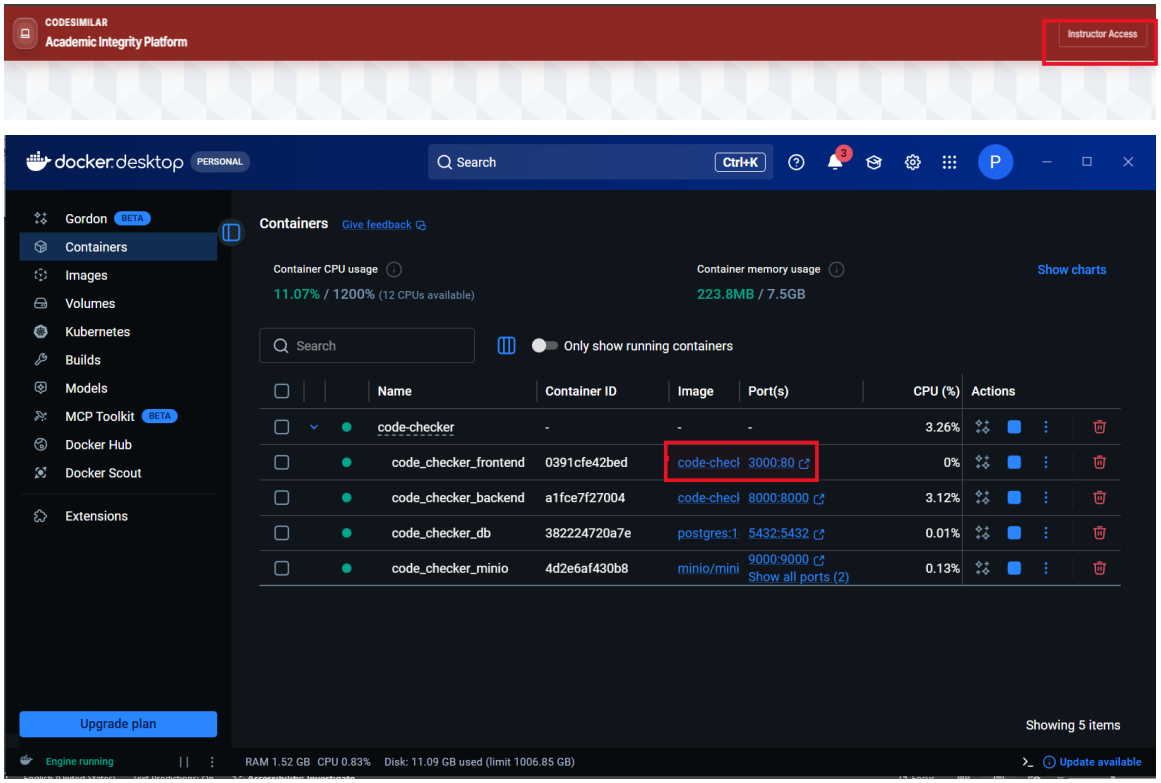
### 3.2.2 Starting the Local Stack

1. Launch Docker Desktop and confirm it is running (the Docker icon should appear in the system tray or menu bar).
2. Open a terminal and navigate to the root of the CodeSimilar source code directory.
3. Run the following command to build and start all services:

```
docker compose up --build
```

4. Once the build completes, open Docker Desktop and click the link shown next to the running container to open the application in your browser.

***NOTE:*** *Google Sign-In is not available when running locally. Use the manual account creation option or the following test credentials: Username: faculty@brocku.ca | Password: password123*




## 3.3 Instructor Account Setup

Two registration methods are available on the hosted platform:

| Method | When to Use |
|---|---|
| Sign in with Google | Preferred method for instructors with a Google-linked institutional or personal email. Instant access with no separate password required. |
| Create One (manual) | Use if you do not have a Google account or prefer a separate password. Requires providing a name, email address, and password. |

### 3.3.1 Signing In with Google

1. Click Instructor Access on the top-right of the landing page.
2. Click Sign in with Google.
3. Select or enter your Google account credentials. You will be redirected to the instructor dashboard on success.

### 3.3.2 Creating a Manual Account

1. Click Instructor Access on the top-right of the landing page.
2. Click Create One below the login form.
3. Complete the registration form with your name, email address, and a password.

4. Submit the form. You will be redirected to the instructor dashboard.

## 3.4 Browser & System Requirements

| Requirement | Details |
|---|---|
| Supported browsers | Google Chrome 110+, Mozilla Firefox 110+, Microsoft Edge 110+ |
| Internet connection | Required for hosted deployment. Not required for local deployment after initial build. |
| Screen resolution | Minimum 1280 x 720 recommended for the dashboard view. |
| No mobile app | CodeSimilar is a desktop-oriented web application. Mobile browsers are not officially supported. |

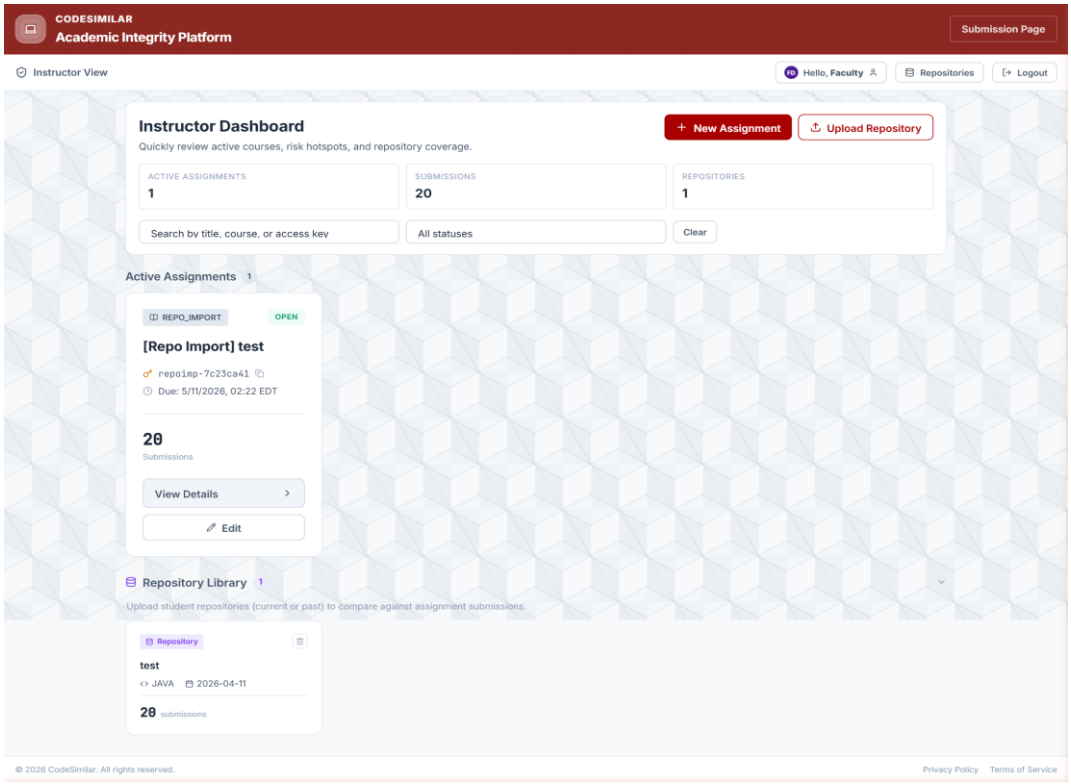
# 4. Instructor Guide

## 4.1 Dashboard Walkthrough

After signing in, instructors are taken to the main dashboard. The dashboard is organised as follows:

| Dashboard Element | Description |
|---|---|
| Active Assignments panel | Lists all currently active assignments. Each card displays the assignment name, course, due date, submission count, and shortcut buttons for uploading, viewing, and analysis. |
| Repositories panel | Lists all uploaded code repositories that are not linked to an active assignment. Repositories serve as reference baselines for cross-repository analysis. |
| New Assignment button | Opens the assignment creation form. |
| Upload Repository button | Opens the repository upload form. |
| Archived panel | Displays assignments that have been manually or automatically archived past their due date. |

***NOTE:*** *The left navigation bar becomes visible once you open a specific assignment and contains links to Pairwise Similarities, Analytics Overview, Collusion Cluster Graph, and Similarity Heatmap.*

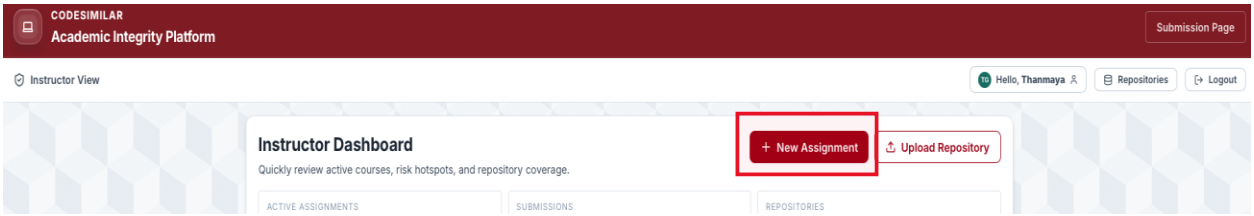


## 4.2 Creating a New Assignment

An assignment represents an active code collection event. Students submit their work against an assignment using the generated access key.

1. Click New Assignment on the instructor dashboard.
2. Fill in the required fields: Assignment Name, Course Name, and Due Date.
3. Click Create & Generate Access Key.
4. The system will generate a unique access key for this assignment (e.g. A1-B2C3). Copy and distribute this key to your students via email, LMS, or any preferred channel.

***NOTE:*** *Each assignment has exactly one access key. Keep this key confidential until you are ready for students to submit. Students cannot submit without a valid key.*

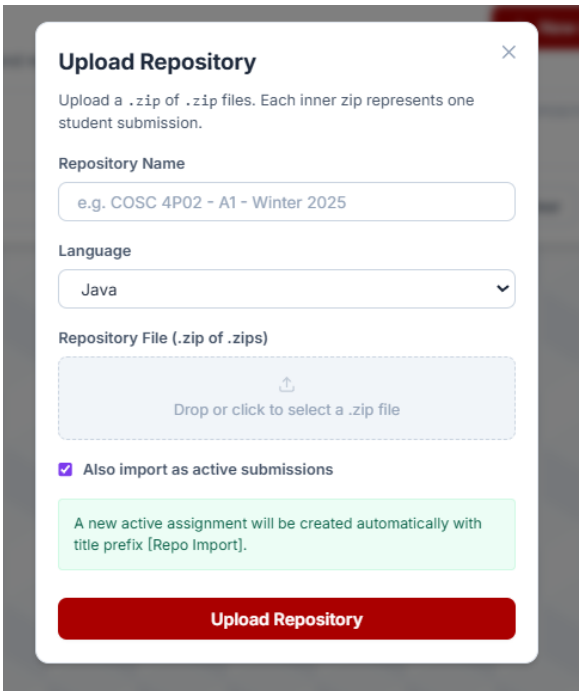



## 4.3 Uploading a Repository

A repository is a collection of code files used as a reference baseline. It can be a previous semester's submissions, sample solutions, or any known codebase you want to compare against. Unlike an active assignment, a repository does not accept student submissions directly.

1. Click Upload Repository on the instructor dashboard.
2. Provide a name and optional description for the repository.
3. Upload a ZIP file or individual source files.
4. By default, uploading a repository also creates an active assignment linked to it. Untick the Create Active Assignment checkbox if you want a standalone reference repository with no associated submission window.
5. Click Upload Repository to confirm.

***TIP:*** *Upload prior semester submissions as repositories to detect recycled work across cohorts during cross-repository analysis.*

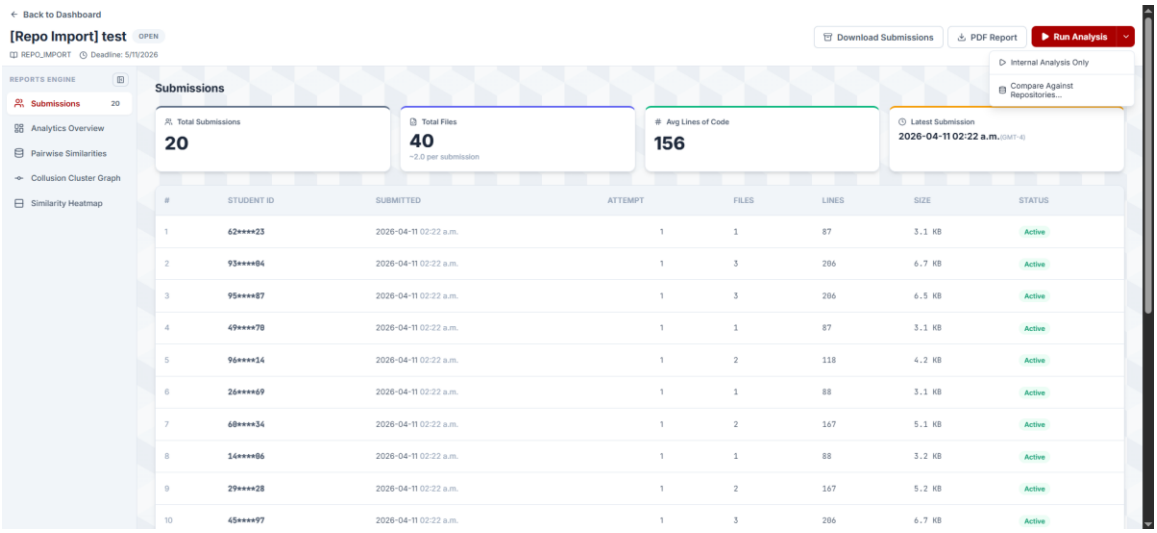



## 4.4 Managing Student Submissions

Once students begin submitting, their entries appear in the assignment's submission list. You can monitor progress and manage files from the assignment detail view.

1. Click View Details on an assignment card from the dashboard.
2. The Submissions tab displays all received submissions, including the student name (anonymised in analysis views), submission timestamp, and file name.
3. Click Download Submissions to download a ZIP archive of all student-submitted files for the assignment.

***NOTE:*** *Student identities are anonymised in all pairwise similarity reports. Instructor-level views retain names for file management purposes only.*

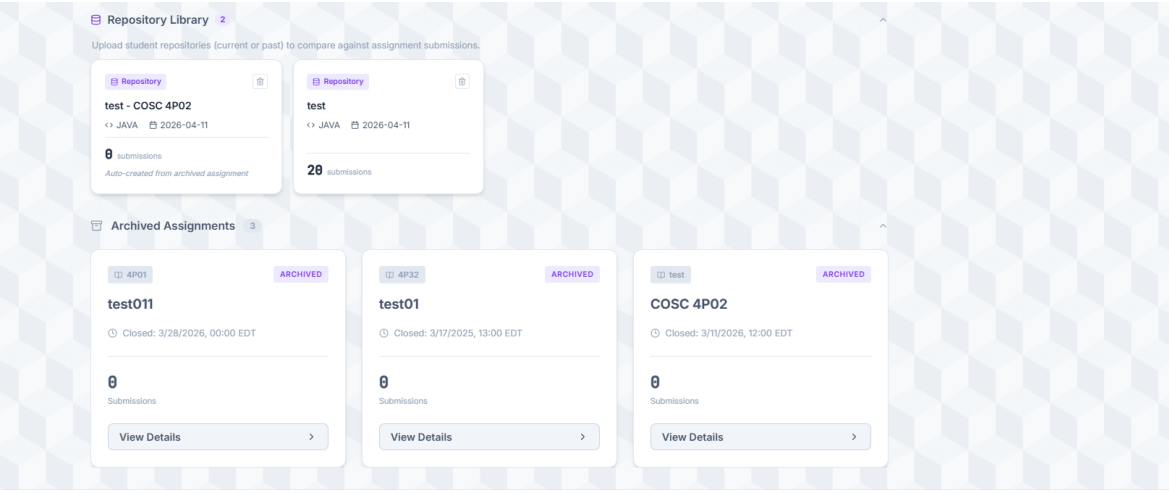


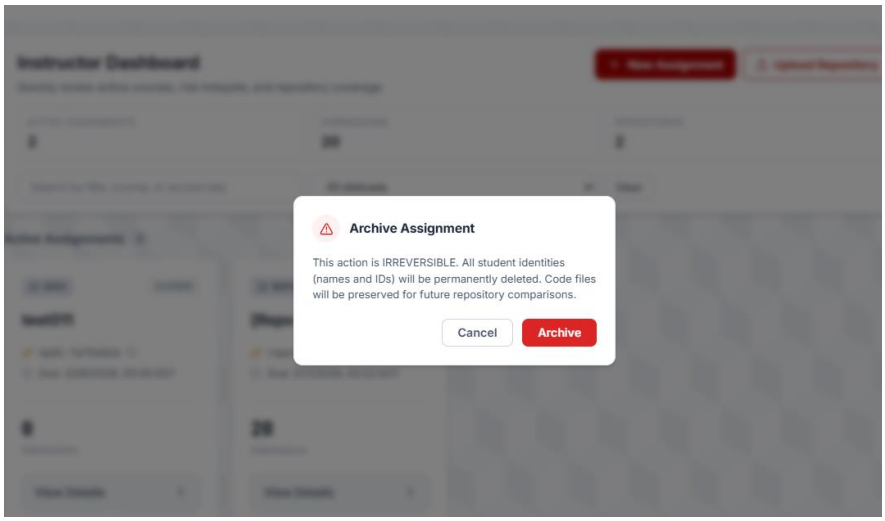

## 4.5 Archive Policy

CodeSimilar applies an automatic archiving policy to manage the lifecycle of active assignments:

| Event | System Behaviour |
|---|---|
| Due date reached | The system prompts the instructor to manually archive the assignment. Submissions are still visible but the assignment is flagged as overdue. |
| 30 days past due date | The assignment is automatically archived and simultaneously added as a repository, making it available for future cross-repository analyses. |
| Manual archive (any time) | The instructor can click the Archive button on the assignment detail page at any time to immediately archive and convert the assignment to a repository. |

***NOTE:*** *Archiving does not delete any submission data. All files and analysis results remain accessible in the Archived panel on the dashboard.*






# 5. Running Similarity Analysis

## 5.1 Internal Analysis

Internal Analysis compares every submitted file against all other submissions within the same active assignment. It is used to detect potential collusion or shared work among students in the same cohort.

1. Click View Details on the target assignment from the dashboard.
2. Click the Run Analysis dropdown button in the top-right area of the assignment view.
3. Select Internal Analysis from the dropdown.
4. The system will process all submissions and populate the Pairwise Similarities panel with results. Depending on the number of submissions, this may take a few moments.

***TIP:*** *Run internal analysis after the submission deadline to ensure all student files are included in the comparison set.*




## 5.2 Cross-Repository Analysis

Cross-Repository Analysis compares the active assignment's submissions against one or more existing repositories. Use this to detect similarities against prior semester work, reference codebases, or known source material.

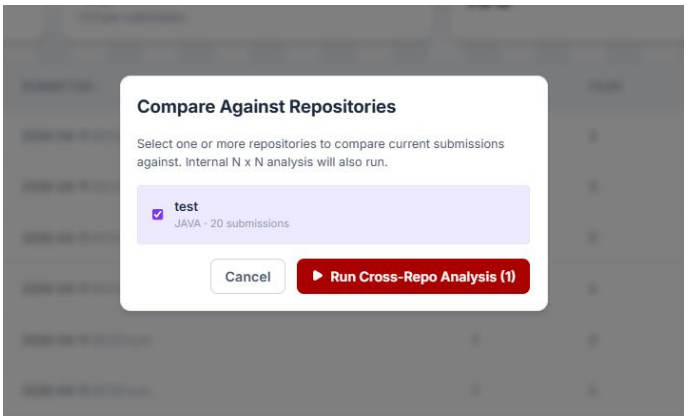
1. Click View Details on the target assignment.
2. Click the Run Analysis dropdown and select Compare Against Repositories.
3. A repository selection panel will appear. Select one or more repositories to compare against. You may select multiple simultaneously.
4. Click Run Cross-Repo Analysis. Results will populate in the Pairwise Similarities panel alongside any prior internal analysis results.

## 5.3 Understanding Similarity Scores

A similarity score represents the percentage of code tokens or structural blocks that two submissions share in common. It is a quantitative indicator, not a verdict. The following guidance assists instructors in contextualising results:

| Score Range | Interpretation | Recommended Action |
|---|---|---|
| 0 % – 30 % | Low similarity. Normal variation expected among independent solutions. | No action required. |
| 31 % – 60 % | Moderate similarity. May reflect shared reference material, common patterns, or partial code reuse. | Review the side-by-side comparison for context before drawing conclusions. |
| 61 % – 85 % | High similarity. Warrants closer manual inspection. | Use Inspect Match to review specific matching blocks. Consider discussing with the student. |
| 86 % – 100 % | Very high similarity. Significant structural or token-level overlap detected. | Treat as a priority case. Document with the PDF report if escalating to an integrity review. |

***NOTE:*** *High similarity scores do not automatically indicate academic dishonesty. Common library usage, provided starter code, and similar algorithmic approaches can all produce elevated scores. Always review the matched code manually before taking any action.*

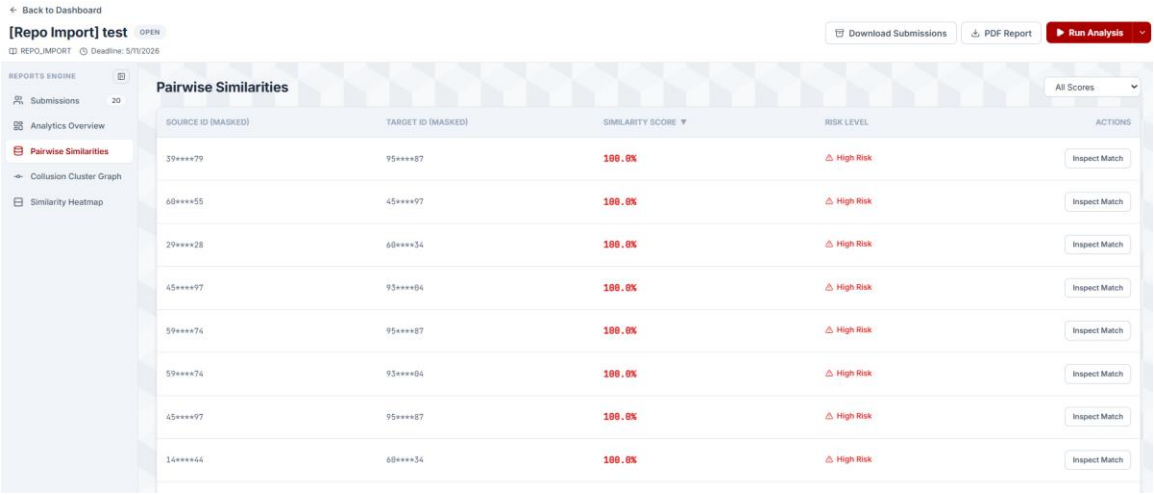


# 6. Reviewing Results

## 6.1 Pairwise Similarities Panel

The Pairwise Similarities panel is the primary results view. It lists all student pairs ranked by similarity score, highest first.

1. After running an analysis, click Pairwise Similarities in the left navigation bar.
2. Each row displays two student identifiers, the similarity score, and an Inspect Match button.
3. Click Inspect Match on any row to open the side-by-side code comparison view for that pair.

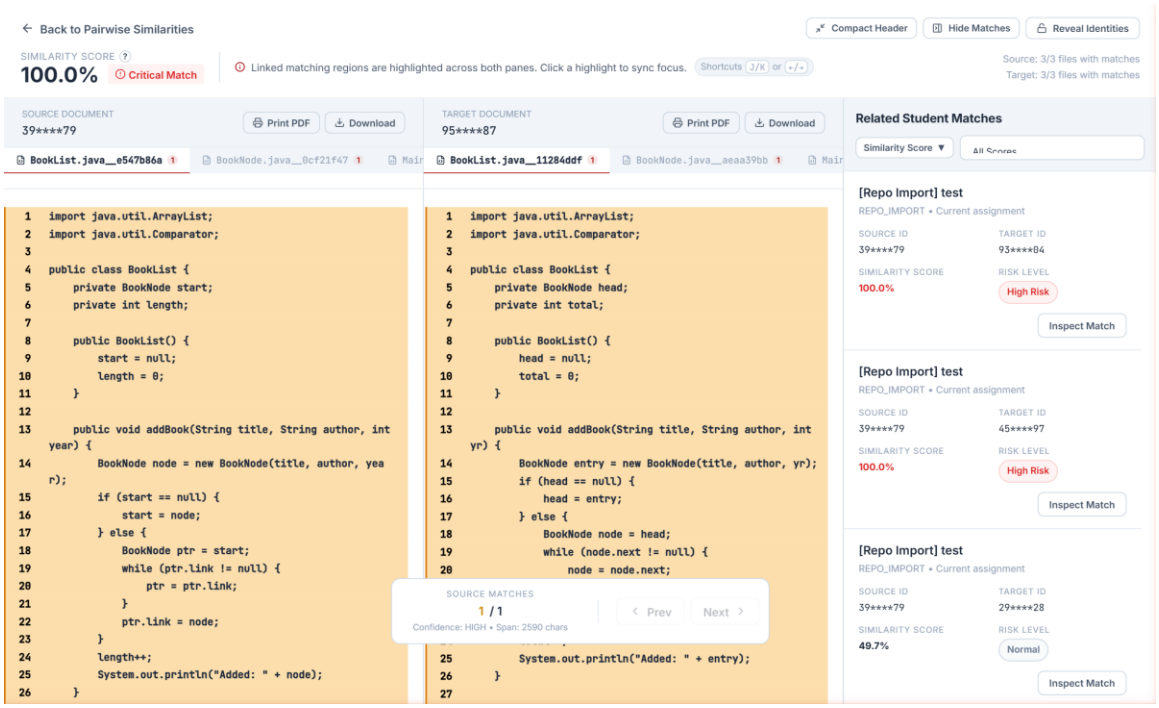



## 6.2 Side-by-Side Code Comparison

The code comparison view displays the two submissions side by side with matching code blocks highlighted. This is the primary tool for determining whether a flagged similarity is meaningful.

1. Within the comparison view, matching code segments are highlighted in both submissions simultaneously.
2. Click Next to navigate to the next matching block between the two submissions.
3. If a student submitted a ZIP file containing multiple source files, use the file selector panel to switch between individual files within the submission.
4. Use the Inspect Match button on the right-side panel to compare the current student against a different student pair without returning to the main list.

5. Click the Download or Print button to save the comparison view with highlighted matches as a document suitable for attaching to an academic integrity report.

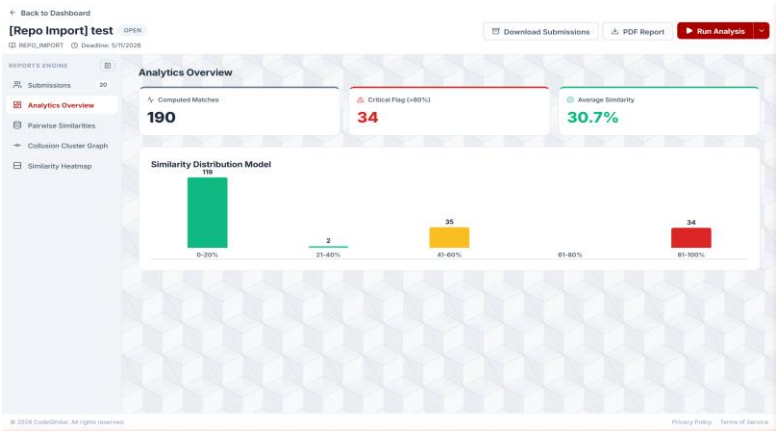



## 6.3 Analytics Overview & Visualisations

Three visualisation tools are available from the left navigation bar once an analysis has been run. Each provides a different perspective on the similarity data across the full submission set

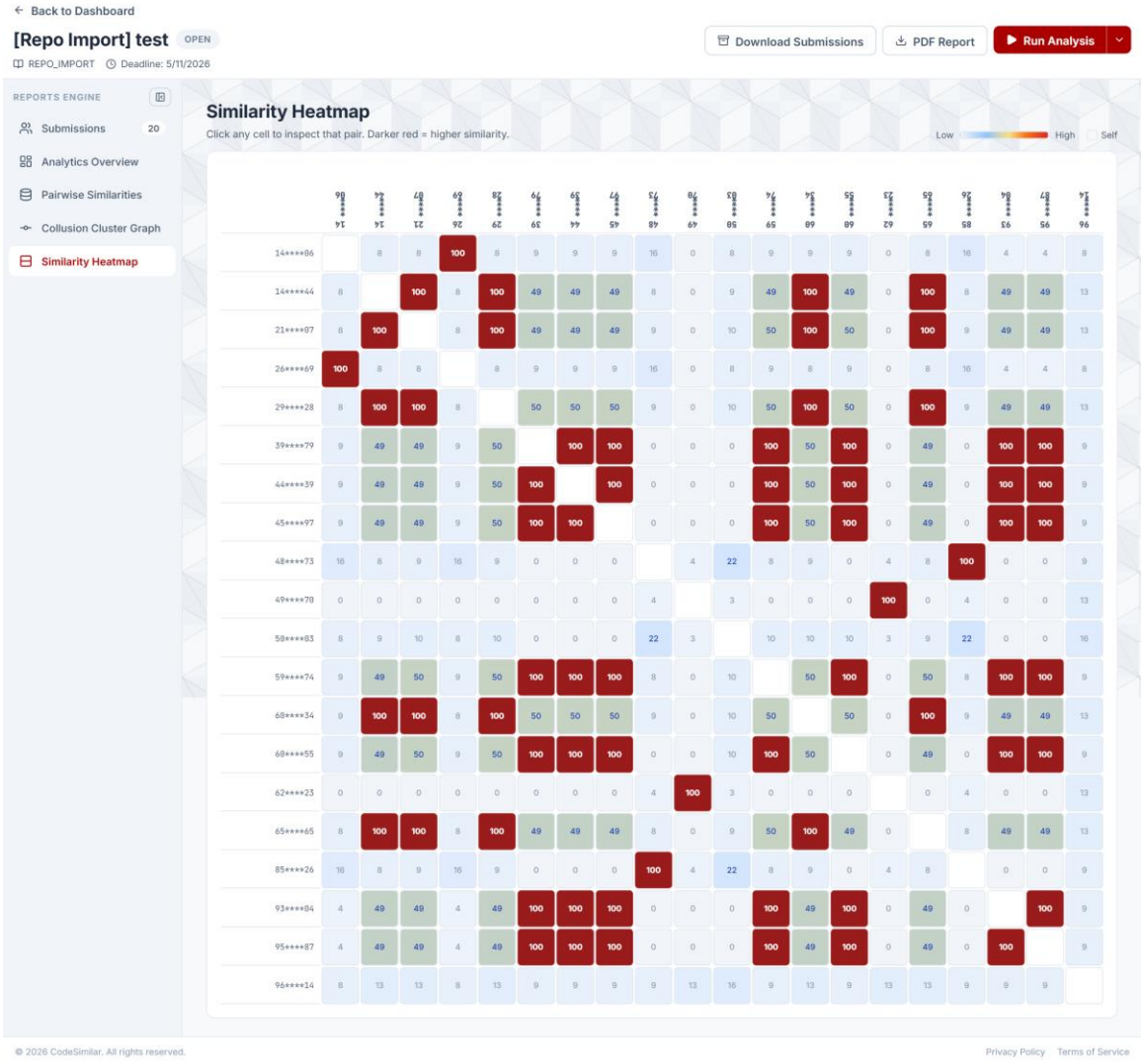
### 6.3.1 Analytics Overview — Bar Graph

The bar graph displays the distribution of similarity scores across all student pairs. The horizontal axis represents score ranges (e.g. 0–10 %, 10–20 %) and the vertical axis shows how many pairs fall within each range. A right-skewed distribution (most pairs at low scores with few at high scores) is typical of honest cohorts. A concentration of pairs in higher score ranges may warrant further investigation.



### 6.3.2 Similarity Heatmap

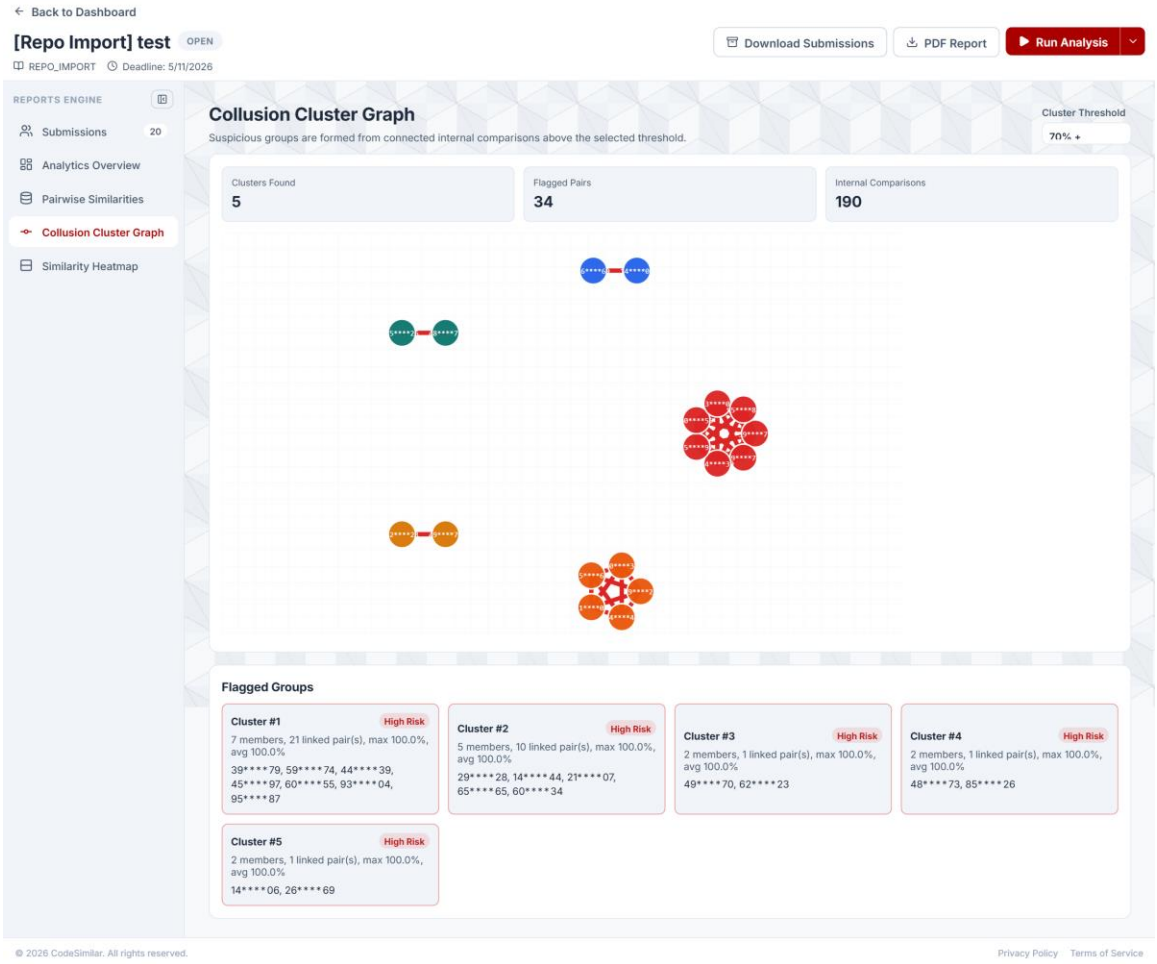
The heatmap is a matrix in which each row and column represents a student submission. The colour intensity of each cell indicates the similarity score between the corresponding pair. Darker cells indicate higher similarity. The heatmap is particularly useful for identifying clusters of students whose work resembles each other, which may be indicative of group collusion.




### 6.3.3 Collusion Cluster Graph

The collusion cluster graph renders the similarity relationships as a network diagram. Each node represents a student submission, and edges connect pairs that exceed a similarity threshold. Tightly grouped clusters of interconnected nodes suggest a network of related submissions. Isolated nodes indicate unique work. This view is best used alongside the pairwise results to identify the scope of a potential integrity concern.

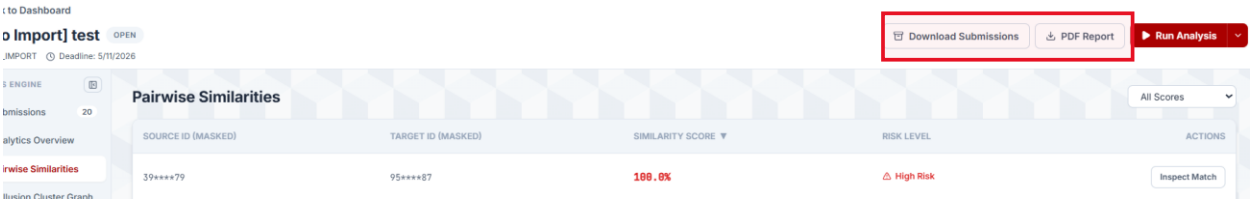
***TIP:*** *Use the collusion cluster graph first to identify which groups of students are involved, then use Inspect Match to examine the specific code overlaps within each identified cluster.*




## 6.4 Exporting Submissions

All active student submissions for an assignment can be downloaded as a single ZIP archive.

1. Click View Details on the assignment.
2. Click Download Submissions.
3. The system will prepare and deliver a ZIP file containing all submitted files, named by student identifier.

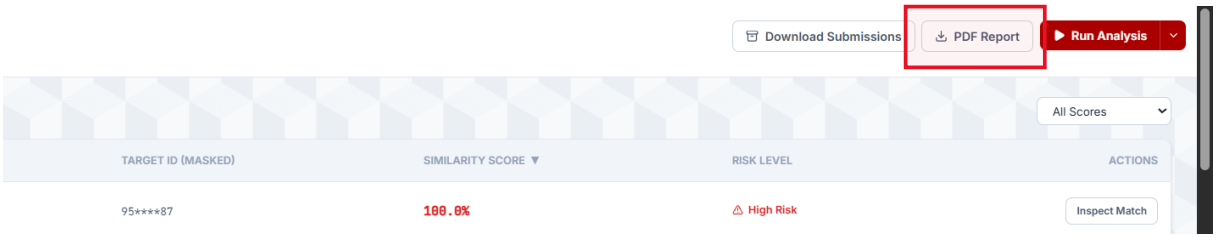


## 6.5 Downloading the PDF Report

A structured PDF report can be generated containing a summary of the analysis results. This report is suitable for attaching to formal academic integrity documentation.

1. From the assignment detail view, click PDF Report.
2. The report is automatically generated and downloaded. It contains the similarity bar graph, the collision matrix, and the similarity heatmap for the full submission set.

***NOTE:*** *The PDF report reflects the state of the analysis at the time of download. If additional submissions are added and the analysis is re-run, download a fresh report to capture the updated results.*



# 7. Student Submission Guide

This chapter is written specifically for students. If you are an instructor or TA, refer to Chapters 3 through 6.

## 7.1 What You Need Before Submitting

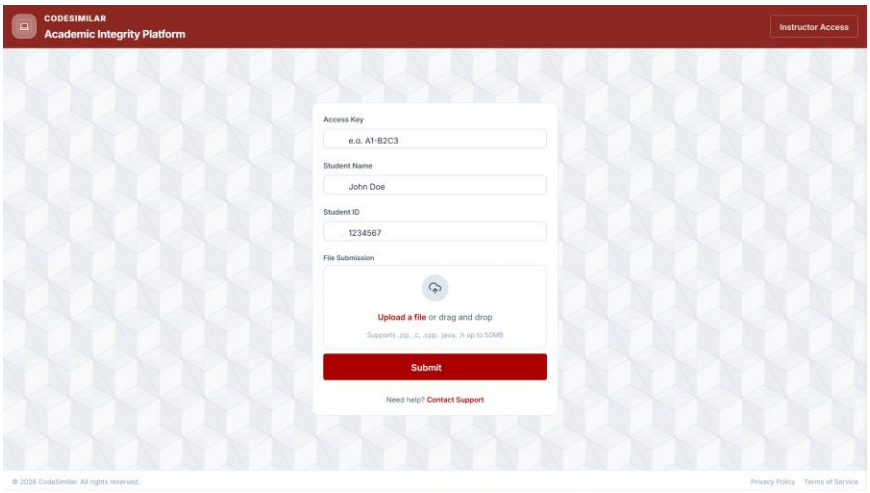
| Item | Details |
|---|---|
| Access Key | A unique code provided by your instructor (e.g. A1-B2C3). You cannot submit without it. |
| Student Name | Your full name as registered with the course. |
| Student ID | Your 7-digit Brock University student identification number. |
| Your Code File | One or more source files in .c, .cpp, .java, or .h format. Multiple files should be bundled into a single .zip archive before uploading. |

> ***NOTE:*** *If you do not have an access key, contact your instructor or TA. Do not attempt to submit without a valid key.*

## 7.2 Submitting Your Work

3. Open a supported web browser and navigate to:

   https://codesimilar.com
4. The submission form is displayed on the landing page. You do not need to create an account.
5. Enter your Access Key in the first field.
6. Enter your Student Name and Student ID.
7. Click Upload a file (or drag and drop your file into the file area) to attach your submission.
8. Click Submit to complete your submission.
9. A confirmation message will appear on screen. No email confirmation is sent, so note the on-screen confirmation before closing your browser.




## 7.3 File Preparation Guidelines

- If your assignment consists of a single file, upload it directly (.c, .cpp, .java, or .h).
- If your assignment consists of multiple files, compress them into a single .zip archive before uploading.
- Do not include compiled binaries, executables, or object files in your submission. Source files only.
- Ensure your file does not exceed 50 MB. Submissions exceeding this limit will be rejected.
- Use your actual student name and ID. Submissions with mismatched credentials may not be attributed correctly.

## 7.4 After Submission

Once submitted, your code is stored securely and will be included in the similarity analysis run by your instructor. The following points apply to all student submissions:

- Your submission is anonymised in all instructor-facing similarity reports. Your name is not visible alongside any similarity score.
- You may not edit or delete a submission after it has been uploaded. If you need to re-submit, contact your instructor to discuss options.
- Submitting work that is not your own, or that has been substantially shared with another student, may result in elevated similarity scores that flag your submission for instructor review.

# 8. Frequently Asked Questions

## 8.1 For Instructors & TAs

### Can I run the analysis more than once for the same assignment?

Yes. The Run Analysis action can be triggered multiple times. Each run processes the current set of submissions, including any new submissions received since the last analysis. Previous results are overwritten by the most recent run.

### Can I compare submissions against multiple repositories at the same time?

Yes. During cross-repository analysis, you may select multiple repositories simultaneously before clicking Run Cross-Repo Analysis. Results from all selected repositories will appear in the Pairwise Similarities panel.

### What happens to an assignment after the due date?

The system will display a prompt inviting you to archive the assignment manually. If no action is taken within 30 days of the due date, the assignment is archived automatically and added as a repository for future reference.

### Will archiving an assignment delete the student submissions?

No. Archiving changes the assignment status and converts it to a reference repository. All submitted files and analysis results remain accessible in the Archived panel on the dashboard.

### Why are some similarity scores higher than expected for clearly original work?

Common causes of elevated scores include: use of provided starter code or code templates, adherence to common algorithmic patterns taught in class, use of standard library boilerplate, or identical comment blocks copied from the assignment specification. Review the matched code blocks manually before drawing any conclusions.

### Can I use CodeSimilar on a mobile device?

The platform is designed for desktop browsers. While the submission portal may function on mobile, the instructor dashboard and analysis views are optimised for a screen resolution of at least 1280 x 720 pixels.

## 8.2 For Students

### Where do I get my access key?

Your access key is provided by your course instructor or TA. It is typically distributed via the course LMS (e.g. Brightspace), email, or in class. If you have not received a key, contact your instructor directly.

### Can I submit more than once?

Submitting multiple times is technically possible if the assignment window is still open. However, all submissions under the same student ID will appear in the system. If you need to replace a previous submission, contact your instructor or TA. Do not submit duplicate files unnecessarily.

### Will the instructor see my name next to my similarity score?

No. Student identities are anonymised in all similarity reports. Your instructor will see a similarity score attributed to an anonymised identifier, not your name. Instructor-level file management views may retain names for submission tracking, but these are separate from the analysis results.

### What file formats are accepted?

The system accepts .c, .cpp, .java, and .h source files. If you have multiple files, compress them into a single .zip archive. Compiled binaries and executables are not accepted.

### I submitted the wrong file. What should I do?

Contact your instructor or TA immediately and explain the situation. Students cannot edit or delete their own submissions through the platform. The instructor will advise on the appropriate next step.

# 9. Support & Contact

## 9.1 Contacting the Development Team

All support requests should be directed to the CodeSimilar development team via email. There is no live chat or phone support at this time.

`pp21qh@brocku.ca`

When submitting a support request, please include the following information to help the team respond efficiently:

- Your name and role (instructor, TA, or student).
- The URL or page where the issue occurred.
- A description of what you attempted to do and what happened instead.
- Any error messages displayed on screen (copy the exact text if possible).
- The browser and operating system you are using.

## 9.2 Known Limitations

| Limitation | Details |
|---|---|
| Google Sign-In | Only available on the hosted deployment (codesimilar.com). Not functional when running locally. |
| Mobile support | The instructor dashboard is not optimised for mobile browsers. Use a desktop browser for full functionality. |
| Resubmission | Students cannot self-manage or delete submissions. Instructor intervention is required. |
| Supported languages | Currently limited to C, C++, Java, and associated header files. Other languages are not analysed. |
| Concurrent analysis runs | Running multiple analyses simultaneously on the same assignment may produce unpredictable results. Complete one run before starting another. |

## 9.3 Reporting a Bug

If you encounter a defect or unexpected behaviour not described in this manual, report it by emailing pp21qh@brocku.ca with the subject line: Bug Report — CodeSimilar. Include the steps to reproduce the issue, the expected outcome, and the actual outcome observed.

# 10. Glossary

The following terms are used throughout this manual and within the CodeSimilar interface.

| Term | Definition |
|---|---|
| Access Key | A unique alphanumeric code generated per assignment (e.g. A1-B2C3) that students must enter to submit their work. Issued by the instructor. |
| Active Assignment | An assignment that is currently open for student submissions. Active assignments appear on the instructor dashboard and are included in analysis runs. |
| Anonymisation | The process of replacing student identifying information with neutral identifiers in similarity reports, protecting student privacy during instructor review. |
| Archive | The act of closing an assignment to new submissions and converting it into a reference repository. Archived assignments remain accessible but are no longer active. |
| Collusion Cluster Graph | A network diagram showing similarity relationships between student submissions. Nodes represent students; edges connect pairs with similarity above a threshold. Clusters suggest groups of related submissions. |
| Cross-Repository Analysis | A similarity analysis mode that compares active assignment submissions against one or more existing reference repositories, rather than only against each other. |
| Internal Analysis | A similarity analysis mode that compares all submissions within a single active assignment against each other. |
| Pairwise Similarity | A similarity score between two specific submissions, expressed as a percentage. Listed in the Pairwise Similarities panel after an analysis run. |
| Repository | A stored collection of code files used as a reference baseline for cross-repository analysis. Can be created by uploading files directly or by archiving an active assignment. |
| Similarity Heatmap | A matrix visualisation in which cell colours represent the similarity score between each pair of student submissions. Darker cells indicate higher similarity. |
| Similarity Score | A numerical percentage (0 %–100 %) indicating how structurally or token-similar two code submissions are to each other. |
| ZIP Archive | A compressed file container (.zip) used to bundle multiple source files into a single upload. Students with multi-file submissions should use this format. |

*End of User Manual*

# CodeSimilar

# SOURCE CODE DOCUMENTATION

## Source Code Documentation

Module & Component Technical Reference

Brock University | 2026

COSC 4P02

Group 8

**Installation Manual** ..... **8**

**1. Introduction** ..... **9**

1.1 Purpose ..... 9
1.2 Scope ..... 9
1.3 Intended Audience ..... 9
1.4 Prerequisites ..... 9
1.5 How to use this document ..... 9

**2. System Overview** ..... **10**

2.1 System Architecture ..... 10
*2.1.1 Frontend (User Interface)* ..... *10*
*2.1.2 Backend API* ..... *10*
*2.1.3 Similarity Analysis Engine* ..... *10*
*2.1.4 Database (Relational Database)* ..... *11*
*2.1.5 File Storage System* ..... *11*
2.2 System Interaction Flow ..... 11
2.3 Deployment Model ..... 12
..... 12

**3. System Requirements** ..... **13**

3.1 Hardware Requirements ..... 13
3.2 Software Requirements ..... 13
3.3 Environment Configuration ..... 13
3.4 Database Setup ..... 14
3.5 Ports and Accesses ..... 14

**4. Installation Methods** ..... **15**

4.1 Hosted Deployment ..... 15
4.2 Local Deployment ..... 15
4.3 Verify Installation ..... 16

**5. Security and Privacy Considerations** ..... **17**

**6. Basic Installation Issues** ..... **18**

**User Manual** ..... **19**

**1. Introduction** ..... **20**

1.1 Purpose of This Manual ..... 20
1.2 Intended Audience ..... 20
1.3 What is CodeSimilar? ..... 20
1.4 How to Use This Manual ..... 20
1.5 Getting Help ..... 21

**2. System Overview** ..... **22**

2.1 Core Workflow ..... 22
2.2 Key Features ..... 22
2.3 Supported File Types ..... 22
2.4 Deployment Options ..... 23

**3. Getting Started ..... 24**
3.1 Accessing the Hosted Platform ..... 24
3.2 Running the Platform Locally ..... 24
*3.2.1 Prerequisites ..... 24*
*3.2.2 Starting the Local Stack ..... 24*
3.3 Instructor Account Setup ..... 25
*3.3.1 Signing In with Google ..... 25*
*3.3.2 Creating a Manual Account ..... 25*
3.4 Browser & System Requirements ..... 26
**4. Instructor Guide ..... 27**
4.1 Dashboard Walkthrough ..... 27
4.2 Creating a New Assignment ..... 28
4.3 Uploading a Repository ..... 28
4.4 Managing Student Submissions ..... 29
4.5 Archive Policy ..... 30
**5. Running Similarity Analysis ..... 31**
5.1 Internal Analysis ..... 31
5.2 Cross-Repository Analysis ..... 31
5.3 Understanding Similarity Scores ..... 32
**6. Reviewing Results ..... 33**
6.1 Pairwise Similarities Panel ..... 33
6.2 Side-by-Side Code Comparison ..... 33
6.3 Analytics Overview & Visualisations ..... 34
*6.3.1 Analytics Overview — Bar Graph ..... 34*
*6.3.2 Similarity Heatmap ..... 35*
*6.3.3 Collusion Cluster Graph ..... 35*
6.4 Exporting Submissions ..... 36
6.5 Downloading the PDF Report ..... 37
**7. Student Submission Guide ..... 38**
7.1 What You Need Before Submitting ..... 38
7.2 Submitting Your Work ..... 38
7.3 File Preparation Guidelines ..... 39
7.4 After Submission ..... 39
**8. Frequently Asked Questions ..... 40**
8.1 For Instructors & TAs ..... 40
*Can I run the analysis more than once for the same assignment? ..... 40*
*Can I compare submissions against multiple repositories at the same time? ..... 40*
*What happens to an assignment after the due date? ..... 40*
*Will archiving an assignment delete the student submissions? ..... 40*
*Why are some similarity scores higher than expected for clearly original work? ..... 40*
*Can I use CodeSimilar on a mobile device? ..... 40*
8.2 For Students ..... 41
*Where do I get my access key? ..... 41*

*Can I submit more than once?* ..... *41*
*Will the instructor see my name next to my similarity score?* ..... *41*
*What file formats are accepted?* ..... *41*
*I submitted the wrong file. What should I do?* ..... *41*

**9. Support & Contact** ..... **42**
9.1 Contacting the Development Team ..... 42
9.2 Known Limitations ..... 42
9.3 Reporting a Bug ..... 42

**10. Glossary** ..... **43**

**Source Code Documentation** ..... **44**

**1. System Overview** ..... **51**
Technology Stack ..... 51

**2. Backend : Core Infrastructure** ..... **51**
config.py ..... 51
*Settingsclass* ..... *51*
*get_settings() → Settings* ..... *52*
database.py ..... 52
*get_db() → Generator[Session, None, None]* ..... *52*
security.py ..... 52

**3. Backend : Analysis Engine** ..... **52**
tokenizer.py ..... 52
fingerprint.py ..... 53
scorer.py ..... 53
*calculate_similarity Return Value* ..... *54*

**4. Backend : API Layer** ..... **54**
dependencies.py ..... 54
*get_current_user(...) → Instructor* ..... *54*
analysis.py ..... 54
auth.py ..... 55
student.py ..... 55
repository.py ..... 56
health.py ..... 56
*health_check() - GET /api/v1/health* ..... *56*

**5. Backend : ORM Models** ..... **56**

**6. Backend : Repositories (Data Access Layer)** ..... **57**
base_repository.py - BaseRepository (generic class) ..... 57
Domain Repositories ..... 57

**7. Backend : Schemas (Validation Layer)** ..... **57**
Schema Pattern (all domains) ..... 57
Notable Schemas ..... 58

**8. Backend : Services (Business Logic)** ..... **58**

analysis_service.py : AnalysisService ..... 58
archival_service.py ..... 58
encryption_service.py : EncryptionService ..... 58
sanitization_service.py : SanitizationService ..... 59
storage_service.py : StorageService ..... 59

**9. Backend : Application Entry Point ..... 59**

**10. Frontend : Application Root ..... 60**

frontend/src/main.tsx ..... 60
frontend/src/App.tsx ..... 60

**11. Frontend : Pages ..... 60**

InstructorLogin.tsx ..... 60
InstructorRegister.tsx ..... 60
InstructorProfile.tsx ..... 60
Dashboard.tsx ..... 61
AssignmentDetails.tsx ..... 61
ComparisonView.tsx ..... 61
Repositories.tsx ..... 61
StudentPortal.tsx ..... 62
Other Pages ..... 62

**12. Frontend : Assignment Details Components ..... 62**

assignmentDetailsUtils.ts ..... 62
SimilarityMatrixTab.tsx ..... 62
Other Assignment Detail Components ..... 63

**13. Frontend : Comparison View Components ..... 63**

comparisonUtils.ts ..... 63
useComparisonData.ts ..... 63
Other Comparison Components ..... 64

**14. Frontend : Dashboard Components ..... 64**

**15. Frontend : Student Portal Components ..... 65**

studentPortalUtils.ts ..... 65
submissionReceiptPdf.ts ..... 65
Other Student Portal Components ..... 65

**16. Frontend : UI Primitives ..... 66**

**17. Frontend : Custom Hooks ..... 66**

useAssignmentDetailsData.ts ..... 66
useDashboardData.ts ..... 66
useFileDrop.ts ..... 66
useStudentPortalSubmission.ts ..... 66

**18. Frontend : Libraries & Utilities ..... 67**

apiClient.ts ..... 67

generateComparisonPDF.ts ..... 67
generatePDFReport.ts ..... 67
**19. Frontend : Type Declarations ..... 68**
auth.ts ..... 68
dashboard.ts ..... 68
jszip.d.ts ..... 68
**20. Frontend : API Client Modules ..... 68**
authApi.ts ..... 68
dashboardApi.ts ..... 68
**21. Module Dependency Map ..... 69**
Backend ..... 69
Frontend ..... 69

# 1. System Overview

The **Code-Checker** system is a web-based academic integrity platform. Students submit source code files through a public-facing portal. Each submission is sanitized and anonymized to protect student identity in compliance with FIPPA, then fingerprinted using a custom-built Winnowing algorithm and compared against other submissions and optional external repositories. Instructors access a secured dashboard to review similarity scores, pairwise comparisons, and collusion cluster analysis.

```
Student → Submit File → Sanitize / Anonymize → Store (Encrypted) → Fingerprint
                                                                        ↓
Instructor → Trigger Analysis → Score Pairs (Jaccard) → View Results Dashboard
```

## Technology Stack

| Layer | Technology |
|---|---|
| Backend API | Python, FastAPI |
| Database ORM | SQLAlchemy |
| Authentication | JWT Tokens + Google OAuth 2.0 |
| Encryption | Fernet symmetric encryption with key rotation |
| Fingerprinting | Custom Winnowing Algorithm (no AI comparison) |
| Frontend | React 18, TypeScript (TSX) |
| PDF Generation | jsPDF (client-side) |
| File Archive | ZIP / blob storage |

# 2. Backend : Core Infrastructure

Contains shared utilities used throughout the entire backend: application settings, database session management, and security primitives (password hashing, JWT generation).

## config.py

Loads and exposes all environment-based application settings using Pydantic's `BaseSettings`. A cached singleton is provided via `get_settings()` to avoid repeated disk reads.

### Settingsclass

| Field | Type | Description |
|---|---|---|
| DATABASE_URL | str | SQLAlchemy-compatible database connection string |
| SECRET_KEY | str | Master secret used for JWT signing |
| ENCRYPTION_KEY | str | Primary Fernet key for encrypting stored file content |
| LEGACY_ENCRYPTION_KEYS | str | Comma-separated older keys for decrypting historical records (key rotation) |
| ACCESS_TOKEN_EXPIRE_MINUTES | int | JWT lifetime in minutes |
| GOOGLE_CLIENT_ID | str | OAuth2 client ID for Google login |

#### get_settings() → Settings

Returns a cached `Settings` instance. Decorated with `@lru_cache` so the `.env` file is parsed only once per process lifetime.

### database.py

Configures the SQLAlchemy engine and session factory. Provides a FastAPI dependency that yields a database session per request and guarantees cleanup in a `finally` block.

#### get_db() → Generator[Session, None, None]

FastAPI dependency generator. Creates a new `Session`, yields it to the route handler, then closes it regardless of success or exception.

```
def my_route(db: Session = Depends(get_db)):
    items = db.query(Submission).all()
    return items
```

### security.py

Pure functions for JWT creation and bcrypt password operations. No side effects; relies only on `app.core.config`.

| Function | Parameters | Returns | Description |
|---|---|---|---|
| create_access_token(data, expires_delta) | data: dict, expires_delta: timedelta\|None | str | Creates a signed JWT. Defaults to ACCESS_TOKEN_EXPIRE_MINUTES when expires_delta is None. |
| verify_password(plain, hashed) | plain_password: str, hashed_password: str | bool | Compares plain text against a stored bcrypt hash. Returns True on match. |
| get_password_hash(password) | password: str | str | Generates a bcrypt hash safe for database storage. |

# 3. Backend : Analysis Engine

### tokenizer.py

Normalizes raw source code into a canonical token stream, removing formatting differences (whitespace, variable names, comments) that would skew similarity scores. Uses the Pygments lexer library internally.

| Function | Parameters | Returns | Description |
|---|---|---|---|
| get_lexer_for_language(language) | language: str | Lexer | Retrieves the Pygments lexer for a given language (e.g. "python", "java", "cpp"). Raises ValueError for unsupported |

| | | | |
|---|---|---|---|
| | | | languages. |
| normalize_source_code (source, language) | source: str, language: str | str | Tokenizes source and maps all identifiers/literals to canonical placeholders (VAR, STR, INT…) so variable renaming cannot defeat similarity detection. |

## fingerprint.py

Implements the **Winnowing Algorithm**. Selects a stable subset of k-gram hashes from a sliding window, producing a fingerprint resilient to insertions, deletions, and code reordering.

| Function | Parameters | Returns | Description |
|---|---|---|---|
| generate_kgrams(tokens, k) | tokens: list[str], k: int | list[tuple] | Produces all k-grams (contiguous subsequences of length k) from the normalized token list. |
| get_stable_hash(kgram) | kgram: tuple | int | Deterministic integer hash for a k-gram. Unlike Python's built-in hash(), consistent across process restarts. |
| winnowing_algorithm(hashes, window_size) | hashes: list[int], window_size: int | list[int] | Slides a window over hash list, selects minimum in each window. Consecutive duplicate minimums are deduplicated. |
| generate_fingerprint(source, language) | source: str, language: str | list[int] | Top-level pipeline: normalize → k-gram → hash → winnow. Returns the full integer fingerprint for a source file. |

## scorer.py

Computes Jaccard similarity scores between two fingerprints and identifies matched code blocks. Includes boilerplate removal to suppress false positives from instructor-provided starter code.

| Function | Visibility | Description |
|---|---|---|
| _block_span(matches) | private | Calculates total token span covered by a list of match objects. |
| _block_confidence(match_count, total) | private | Computes confidence ratio match_count/total clamped to [0.0, 1.0]. |
| _is_generic_or_weak(hashes) | private | Heuristic filter identifying hashes |

| | | |
|---|---|---|
| | | from trivially common patterns (e.g. empty main functions) that should not contribute to scoring. |
| _merge_match_pairs(pairs) | private | Consolidates overlapping or adjacent match pairs into contiguous blocks to reduce noise in comparison output. |
| remove_boilerplate(fingerprint, boilerplate_hashes) | public | Strips hashes present in the boilerplate set from a submission fingerprint before scoring. |
| calculate_similarity(fp_a, fp_b) | public | Core scoring function. Returns Jaccard similarity index and matched hash positions for both submissions. |

### calculate_similarity Return Value

| Key | Type | Description |
|---|---|---|
| score | float | Jaccard index: \|A ∩ B\| / \|A ∪ B\| value between 0.0 (no match) and 1.0 (identical) |
| matched_in_a | list[int] | Hash indices matched within submission A (used for frontend highlighting) |
| matched_in_b | list[int] | Hash indices matched within submission B (used for frontend highlighting) |

# 4. Backend : API Layer

FastAPI route handlers organized under the `/api/v1/` prefix. Each module handles one domain.

## dependencies.py

### get_current_user(...) → Instructor

Validates the Bearer JWT from the `Authorization` header. Decodes the token, retrieves the instructor record from the database, and returns it. Raises `HTTP 401` if the token is invalid or expired. Injected into all protected routes via `Depends(get_current_user)`.

## analysis.py

| Function / Type | Kind | Description |
|---|---|---|
| AnalysisRequest | Pydantic model | Request body for POST /analysis/trigger. Fields: assignment_id (int), repository_ids (list[int]). |
| _score_pairs(submissions, db) | Private | Iterates all pairs within a submission set and calls calculate_similarity on each pair. |

| | | |
|---|---|---|
| _score_cross_pairs(submissions, repo_submissions, db) | Private | Scores each assignment submission against all submissions in the selected external repositories. |
| _build_submission_fingerprint(submission, db) | Private | Fetches stored file content and calls generate_fingerprint after applying boilerplate removal. |
| _build_match_metadata(result, sub_a, sub_b) | Private | Constructs JSON metadata blob with token-level match positions for the frontend visualizer. |
| perform_full_analysis(assignment_id, db) | Public | Executes the complete analysis pipeline synchronously. Called by trigger_analysis. |
| trigger_analysis(request, db, user) | POST/analysis/trigger | Authenticated endpoint. Launches perform_full_analysis and returns a summary of scores generated. |

## auth.py

| Endpoint / Function | Method | Description |
|---|---|---|
| register_instructor(data, db) | POST/auth/register | Creates a new instructor account. Hashes password before storage. |
| login(form_data, db) | POST/auth/login | Validates email/password credentials and returns a JWT access token. |
| google_auth(credential, db) | POST/auth/google | Verifies Google ID token and creates or retrieves the matching instructor account. |
| get_me(current_user) | GET/auth/me | Returns the authenticated instructor's profile data. |
| update_me(data, current_user, db) | PUT/auth/me | Updates instructor display name and avatar. |
| change_password(data, current_user, db) | PUT/auth/change-password | Verifies current password then stores the new bcrypt hash. |

## student.py

Public-facing endpoints - **no authentication required**.

| Function | Method | Description |
|---|---|---|
| get_assignment_info(assignment_key, db) | GET/student/assignment/{key} | Returns non-sensitive assignment metadata (title, deadline, allowed languages) for the student portal display. |
| submit_assignment(key, files, student_id, db) | POST/student/submit | Accepts source file uploads, sanitizes content, encrypts it, stores submission records, and returns a receipt ID. |

### repository.py

| Function | Method | Description |
|---|---|---|
| list_repositories(db, current_user) | GET/repositories | Returns all repositories owned by the authenticated instructor. |
| upload_repository(files, metadata, db, user) | POST/repositories | Accepts a ZIP or set of files and stores them as a new reference repository. |
| delete_repository(repo_id, db, user) | DELETE/repositories/{id} | Removes a repository and all its stored submissions. |

### health.py

#### health_check() - GET /api/v1/health

Verifies database connectivity and storage service availability. Returns `{"status": "ok"}` or raises `HTTP 503` on failure. Used for deployment readiness probes.

# 5. Backend : ORM Models

SQLAlchemy declarative models. All classes inherit from `Base` declared in `base.py`. The `base.py` file itself only declares the shared `declarative_base()` instance with no model definitions of its own.

| Model Class | Table | Description |
|---|---|---|
| Assignment | assignments | An instructor-created task with a submission window and language constraints. |
| Boilerplate | boilerplates | Instructor-uploaded starter code; hashes are excluded from similarity scoring. |
| ComparisonResult | comparison_results | Stores the similarity score and match metadata for a pair of submissions. |
| Course | courses | Logical grouping of assignments under an instructor account. |
| Instructor | instructors | Authenticated user who manages assignments and views analysis results. |
| Repository | repositories | Named collection of reference submissions for cross-repository analysis. |
| RepositorySubmission | repository_submissions | A single file entry within a reference repository. |
| StoredFile | stored_files | Encrypted, stored source file belonging to a student submission. |
| Submission | submissions | A student submission event; links to one or more StoredFile records. |

# 6. Backend : Repositories (Data Access Layer)

Data Access Objects (DAOs) implementing CRUD and query operations. All domain repositories extend `BaseRepository`.

## base_repository.py - BaseRepository (generic class)

| Method | Description |
|---|---|
| get(id) | Fetch a single record by primary key. |
| get_all() | Fetch all records of this model type. |
| create(schema) | Insert a new record from a Pydantic create schema. |
| update(id, schema) | Partially update a record using a Pydantic update schema. |
| delete(id) | Remove a record by primary key. |

## Domain Repositories

| Repository | Model | Notable Extra Methods |
|---|---|---|
| AssignmentRepository | Assignment | get_by_key(), get_active_for_instructor() |
| BoilerplateRepository | Boilerplate | get_by_assignment() |
| ComparisonResultRepository | ComparisonResult | get_by_assignment(), get_pair() |
| CourseRepository | Course | get_by_instructor() |
| InstructorRepository | Instructor | get_by_email(), get_by_username() |
| StoredFileRepository | StoredFile | get_by_submission() |
| SubmissionRepository | Submission | get_by_assignment(), get_by_student_id() |

# 7. Backend : Schemas (Validation Layer)

Pydantic v2 models defining validated request and response shapes for the API layer. All schemas inherit from `ORMBaseModel` which enables direct construction from SQLAlchemy ORM objects via `model_config = ConfigDict(from_attributes=True)`.

## Schema Pattern (all domains)

| Suffix | Purpose |
|---|---|
| *Base | Shared fields common to all variants of this entity. |
| *Create | Fields required when creating a new record (used in POST request bodies). |
| *Update | Optional fields for partial updates (used in PUT/PATCH bodies). |
| *InDB | Internal schema including auto-generated fields |

| | |
|---|---|
| | (id, created_at, etc.). |
| *Response | Safe outbound schema returned to API consumers. |

### Notable Schemas

| Schema | Key Fields |
|---|---|
| InstructorRegister | email, password, display_name |
| GoogleCredential | credential (Google ID token string) |
| TokenResponse | access_token, token_type, instructor (InstructorResponse) |
| SubmissionResponse | id, receipt_id, assignment_id, submitted_at, file_count |
| AnalysisRequest | assignment_id, repository_ids (list[int]) |

# 8. Backend : Services (Business Logic)

Business logic layer between the API routes and the database. Services encapsulate complex multi-step operations so route handlers stay lean.

### analysis_service.py : AnalysisService

| Method | Description |
|---|---|
| fingerprint_submission(submission, db) | Generates a fingerprint for a submission's stored files after fetching their encrypted content. |
| score(fp_a, fp_b, boilerplate) | Calls calculate_similarity after applying remove_boilerplate on both fingerprints. |

### archival_service.py

| Function | Description |
|---|---|
| effective_status(assignment) | Returns current display status based on deadline and state fields. One of: "active", "archived", "overdue", "pending". |
| finalize_archive(assignment_id, db) | Transitions a single assignment to archived status, cleaning up transient data. |
| finalize_overdue_archives(db) | Batch job scanning for overdue active assignments and archiving them. Called by the background scheduler in main.py. Returns count of assignments archived. |

### encryption_service.py : EncryptionService

| Function / Method | Description |
|---|---|
| _derive_fernet_key(raw_key) | Private. Derives a valid 32-byte URL-safe base64 Fernet key from an arbitrary string using SHA-256. |
| _parse_legacy_keys(keys_str) | Private. Parses LEGACY_ENCRYPTION_KEYS |

| | |
|---|---|
| | setting into a list of Fernet instances for decryption fallback. |
| encrypt(data: bytes) | Encrypts raw bytes using the current primary key. |
| decrypt(token: bytes) | Attempts decryption with primary key, then falls back through legacy keys in order. |

### sanitization_service.py : SanitizationService

Processes uploaded source code to remove or replace personal identifiers (student names in comments, student IDs in string literals) before storage. Ensures FIPPA compliance.

| Method | Parameters | Returns | Description |
|---|---|---|---|
| sanitize(source, language) | source: str, language: str | str | Strips PII patterns from source code using language-aware parsing. Returns sanitized source string. |

### storage_service.py : StorageService

| Method | Description |
|---|---|
| write(path, data) | Persists encrypted bytes to the configured storage backend. |
| read(path) | Retrieves file content by path. |
| delete(path) | Removes a stored file. |
| exists(path) | Returns True if a file exists at the given path. |

## 9. Backend : Application Entry Point

Bootstraps the FastAPI application: registers all routers, configures CORS, starts the archive scheduler, and seeds a default instructor on first run.

| Function | Description |
|---|---|
| _seed_default_instructor(db) | Creates a default admin instructor account if no instructors exist. Used for initial system setup. |
| run_archive_scheduler() | Background thread function. Runs finalize_overdue_archives on a periodic interval configured in settings. |
| lifespan(app) | FastAPI lifespan context manager. Handles startup (DB init, seeding, scheduler) and shutdown cleanup. |
| read_root() | GET / - Returns a welcome JSON message confirming the API is running. |

## 10. Frontend : Application Root

### frontend/src/main.tsx

React application entry point. Mounts the `<App>` component inside `<AuthContextProvider>` into the DOM root element.

### frontend/src/App.tsx

| Component / Function | Description |
|---|---|
| GlobalHeader | Renders the top navigation bar. Conditionally hidden on sub-pages that have their own header. Reads auth state from useAuth(). |
| AppContent | Defines the React Router <Routes> tree, mapping URL paths to page-level components. |
| App | Root component. Wraps AppContent with ErrorBoundary and ToastProvider. |

# 11. Frontend : Pages

Route-level components rendered by React Router. Each represents a full page view.

### InstructorLogin.tsx

| Handler | Description |
|---|---|
| handleLogin(e) | Submits email/password to POST /auth/login and stores the returned JWT in auth context. |
| handleGoogleSuccess(credential) | Forwards Google ID token to POST /auth/google for OAuth login. |
| onResize() | Adjusts layout for mobile and tablet viewport changes. |

### InstructorRegister.tsx

| Handler | Description |
|---|---|
| set(field, value) | Updates a specific form field in component state. |
| handleSubmit(e) | Validates and submits the registration form to POST /auth/register. |
| handleGoogleSuccess(credential) | Registers a new instructor account using Google OAuth credential. |

### InstructorProfile.tsx

| Handler | Description |
|---|---|
| setProfile(field, value) | Updates a specific profile form field in state. |
| handleProfileSave() | Submits updated display name / avatar to PUT /auth/me. |
| handlePasswordChange() | Validates and submits new password to PUT /auth/change-password. |
| handleLogout() | Clears JWT from context and redirects to the login page. |

## Dashboard.tsx

| Handler | Description |
|---|---|
| formatDeadline(date) | Converts ISO date strings to human-readable deadline labels. |
| openCreateModal() | Opens AssignmentModal in create mode. |
| openEditModal(assignment) | Opens AssignmentModal pre-populated with existing assignment data. |
| handleArchive(id) / doArchive(id) | Prompts confirmation then sends archive request to the backend. |
| handleCreateAssignment(data) | Submits new assignment form data to the backend. |
| pad(n) | Zero-pads a number to two digits for deadline display formatting. |

## AssignmentDetails.tsx

| Handler | Description |
|---|---|
| handleTabChange(tab) | Switches between Submissions, Pairwise Results, Similarity Matrix, Analytics Overview, and Collusion Cluster tabs. |

## ComparisonView.tsx

Side-by-side code comparison viewer for reviewing two flagged submissions.

| Handler | Description |
|---|---|
| onResize() | Recalculates pane widths when the browser window resizes. |
| scrollToMatch(matchId) | Scrolls both code panes to a specific match block by ID. |
| jumpToBlock(blockIndex) | Navigates directly to a matched block by its index. |
| jumpByDelta(delta) | Moves forward (+1) or backward (-1) through matched blocks. |
| onKeyDown(e) | Keyboard shortcut handler — arrow keys for match navigation. |
| handleDownloadSubmission(side) | Downloads the raw source file for the left or right submission pane. |
| getLineStyle(lineNumber, side) | Returns the CSS highlight class for a line number based on match data. |

## Repositories.tsx

| Handler | Description |
|---|---|
| fetchRepos() | Loads all reference repositories from the backend. |
| handleUpload(files) | Initiates repository file upload to POST /repositories. |
| handleDelete(id) / doDelete(id) | Prompts confirmation then sends a delete request to the backend. |

## StudentPortal.tsx

| Handler | Description |
|---|---|

| | |
|---|---|
| handleFileChange(files) | Validates file types and sizes, stages files for submission. |
| onResize() | Adjusts layout for mobile viewports. |

### Other Pages

| Page | Description |
|---|---|
| NotFound.tsx | 404 fallback page rendered for unmatched routes. No interactive logic. |
| PrivacyPolicy.tsx | Static FIPPA/privacy policy page. No interactive logic. |
| TermsOfService.tsx | Static terms of service page. No interactive logic. |

# 12. Frontend : Assignment Details Components

### assignmentDetailsUtils.ts

| Function | Description |
|---|---|
| shortId(id) | Truncates a UUID to first 8 characters for compact display in tables and labels. |
| studentKey(submission) | Derives a display key for a submission (anonymized if FIPPA privacy mode is active). |
| formatBytes(bytes) | Converts a byte count to human-readable string, e.g. "12.3 KB". |
| filterAndSortComparisons(results, filter) | Filters comparison results by a score threshold and sorts by score descending. |
| buildUniqueMatrixIds(results) | Extracts the sorted unique set of submission IDs present in comparison results for the similarity matrix. |
| getSubmissionsCount(assignment) | Returns total submission count from assignment metadata. |

### SimilarityMatrixTab.tsx

Renders an N×N color-coded grid of similarity scores between all submission pairs.

| Function | Description |
|---|---|
| lerpColor(a, b, t) | Linear interpolation between two hex color strings at position $t \in [0,1]$. |
| getMatrixCellColor(score) | Maps a similarity score [0,1] to a color gradient: green → yellow → red. |
| getTextColor(bgColor) | Returns black or white text color based on background luminance for WCAG readability. |
| getComparison(idA, idB, results) | Looks up the comparison result for a given submission pair from the results array. |

### Other Assignment Detail Components

| Component | Description |
|---|---|
| CollusionClusterTab.tsx | Visualizes groups of submissions with mutual high similarity — potential collusion rings. |
| PairwiseResultsTab.tsx | Tabular list of all comparison pairs sorted by score. Clicking a row navigates to ComparisonView. |
| AnalyticsOverviewTab.tsx | Summary statistics: score distribution histogram, average score, flagged submission count. |
| AssignmentDetailsHeader.tsx | Page header with assignment title, course name, deadline, and action buttons (trigger analysis, archive). |
| AssignmentDetailsSidebar.tsx | Side panel showing assignment metadata and tab navigation quick-links. |
| SubmissionsTab.tsx | Lists all individual submissions with timestamp, file count, and size. |
| RepositorySelectionModal.tsx | Modal for choosing which secondary repositories to include before triggering cross-repo analysis. |

# 13. Frontend : Comparison View Components

### comparisonUtils.ts

| Function | Description |
|---|---|
| buildMaskedIdentity(submission) | Builds a display-safe identity label for a submission for FIPPA-compliant rendering. |
| maskIdentity(name) | Replaces real names with anonymized tokens. |
| getIdentityTokens(comparison) | Extracts the masked label pair from a comparison result object. |
| buildCharToLineMap(source) | Maps character offsets to line numbers for match highlight rendering. |
| markRange(map, start, end) | Marks a character range as highlighted in the line map. |
| synthesize(ranges, source) | Converts match ranges into annotated line objects for rendering in CodePane. |
| countFileMatches(comparison, fileIndex) | Counts matched blocks in a specific file tab. |
| filterAndSortOtherMatches(matches, currentId) | Filters out the current submission from the related-matches sidebar list. |

### useComparisonData.ts

Custom React hook fetching and preparing all data needed for the ComparisonView page.

| Function | Description |
|---|---|
| fetchComparison(id) | Loads comparison result including source files and match metadata from the backend. |
| fetchOtherAssignmentMatches(submissionId) | Loads cross-assignment matches for the related-matches sidebar panel. |

### Other Comparison Components

| Component | Description |
|---|---|
| CodePane.tsx | Renders one side of the comparison with syntax-highlighted, match-annotated source code. handlePrintPDF() triggers client-side PDF export via generateComparisonPDF. |
| FileTabBar.tsx | Tab navigation for switching between multiple source files within a submission. |
| FloatingMatchNavigator.tsx | Floating panel with prev/next arrows and a match counter for navigating between matched blocks. |
| RelatedMatchesSidebar.tsx | Lists other submissions sharing matches with the current one. getStatusStyles(status) maps a status string to CSS severity indicator classes. |

# 14. Frontend : Dashboard Components

| Component | Description |
|---|---|
| ActiveAssignmentCard.tsx | Card for a live assignment showing submission count, deadline, and analysis status. handleCopyKey() copies the student submission portal link to clipboard. |
| ArchivedAssignmentsSection.tsx | Collapsible section listing completed or archived assignments with access to historical results. |
| AssignmentModal.tsx | Create/edit assignment form modal. Fields: title, deadline, allowed languages, boilerplate upload. |
| DashboardHeaderStats.tsx | KPI strip showing total submissions, active assignments, and flagged pairs count. |
| FileDropInput.tsx | Drag-and-drop file input area wrapping the useFileDrop hook for reusable file selection. |
| RepositoriesSection.tsx | Lists available reference repositories with upload and delete action buttons. |
| RepositoryUploadModal.tsx | Modal for uploading a new reference repository (repository name + source files). |

# 15. Frontend : Student Portal Components

## studentPortalUtils.ts

| Function | Description |
|---|---|
| formatTorontoTimestamp(iso) | Converts an ISO timestamp to Toronto (Eastern) timezone display format. |
| formatKb(bytes) | Converts a byte count to a kilobytes display string. |

## submissionReceiptPdf.ts

Generates a downloadable PDF receipt for the student after a successful submission using jsPDF.

| Function | Description |
|---|---|
| slugify(text) | Converts text to a filename-safe slug (lowercase, hyphens). |
| drawHeader(doc, data) | Renders the PDF header with system branding. |
| drawFooter(doc, pageNum) | Renders the page footer with page number. |
| drawSectionTitle(doc, title, y) | Draws a section heading row at the given y-coordinate. |
| drawStatusStrip(doc, status, y) | Renders a colored status banner strip. |
| drawSimpleInfoTable(doc, rows, y) | Renders a two-column info table in the PDF. |
| buildSubmissionReceiptPdf(data) | Orchestrates the full PDF layout and returns a downloadable Blob. |

## Other Student Portal Components

| Component | Description |
|---|---|
| StudentSubmissionForm.tsx | Main upload form with assignment info, file drop area, student ID field, and submit button. |
| StudentSubmissionSuccess.tsx | Confirmation screen after a successful submission. handleCopyReceiptId() copies the receipt ID to clipboard. |
| types.ts | TypeScript type definitions for student portal data (assignment info, submission result, receipt data). |

# 16. Frontend : UI Primitives

| Component | Description |
|---|---|
| ConfirmModal.tsx | Generic confirmation dialog with confirm and cancel action buttons. |
| ErrorBoundary.tsx | React class component that catches render errors and displays a graceful fallback UI instead of a blank screen. |
| LoadingSpinner.tsx | Animated spinner displayed during async data fetches. |
| Toast.tsx | Toast notification system. useToast() hook exposes show(message, type). dismiss(id) removes a toast by ID. |

# 17. Frontend : Custom Hooks

## useAssignmentDetailsData.ts

Fetches all data required for the AssignmentDetails page: assignment metadata, submissions list, and comparison results.
**Returns:** `{ assignment, submissions, comparisons, loading, error, refetch }`

`onResize()` — throttled handler that triggers layout recalculations on window resize.

## useDashboardData.ts

Fetches all data for the Dashboard page: courses, active assignments, archived assignments, and repositories.
**Returns:** `{ courses, activeAssignments, archivedAssignments, repositories, loading, refetch }`

`init()` — internal async function called on mount and after any mutation to reload all dashboard data.

## useFileDrop.ts

Reusable drag-and-drop file input logic used by AssignmentModal, RepositoryUploadModal, and StudentSubmissionForm.
**Returns:** `{ files, isDragging, inputProps, dropProps, reset }`

`isAccepted(file)` — checks whether a dropped file's extension matches the configured allowed list.

## useStudentPortalSubmission.ts

Orchestrates the full student submission flow: file validation → API call → receipt PDF generation.
**Returns:** `{ submitFiles, status, receiptData, error }`

# 18. Frontend : Libraries & Utilities

## apiClient.ts

Configures the base Axios instance with the backend API URL and automatic `Authorization: Bearer <token>` header injection. All API calls throughout the application use this shared client to ensure consistent authentication handling.

## generateComparisonPDF.ts

Generates a downloadable side-by-side comparison PDF for an instructor reviewing a flagged pair.

| Function | Description |
|---|---|
| buildLineHighlightMap(lines) | Builds a map of line numbers to their highlight colors for match visualization. |
| drawPageHeader(doc, meta) | Renders the page header with assignment and submission identifiers. |
| drawFileHeader(doc, filename, y) | Draws a section separator for each source file within the comparison. |
| renderFile(doc, lines, highlights, x, y) | Renders annotated source code into the PDF at given x/y coordinates. |
| generateComparisonPDF(data) | Orchestrates the full two-column PDF layout and triggers the browser download. |

## generatePDFReport.ts

Generates the instructor-facing analytics PDF report for an entire assignment.

| Function | Description |
|---|---|
| slugify(text) | Converts text to a filename-safe slug. |
| formatDateTime(iso) | Converts an ISO timestamp to locale date-time string. |
| formatPercentage(score) | Converts a [0,1] float to a percentage string. |
| maskIdentity(name) | Anonymizes a student display name for FIPPA compliance in the report. |
| getStatusStyle(score) | Returns severity label and color for a given similarity score. |
| drawSectionTitle(doc, title, y) | Renders a section heading in the PDF. |
| drawPill(doc, text, x, y, color) | Renders a color-coded pill/badge label. |
| drawMetricCard(doc, label, value, x, y) | Renders a single KPI metric card in the report layout. |

# 19. Frontend : Type Declarations

## auth.ts

TypeScript interfaces for authentication data. Also contains display helper functions:

| Function | Description |
|---|---|
| formatDisplayName(instructor) | Returns the instructor's first name or username for compact UI display. |
| getInitials(name) | Extracts 1–2 letter initials from a display name for avatar rendering. |
| getAvatarColor(name) | Deterministically maps a name string to one of several avatar background colors. |

## dashboard.ts

TypeScript interfaces for all dashboard data structures: `Course`, `Assignment`, `Repository`, `DashboardStats`. Contains only type declarations with no runtime functions.

## jszip.d.ts

TypeScript ambient module declaration for the `jszip` library, enabling type-safe client-side ZIP file creation in the browser.

# 20. Frontend : API Client Modules

## authApi.ts

Typed wrapper functions over the shared apiClient for all authentication endpoints: `login()`, `register()`, `googleAuth()`, `getMe()`, `updateMe()`, `changePassword()`. Each function handles request construction and returns typed response data.

## dashboardApi.ts

Typed wrapper functions for all dashboard data endpoints.

`buildAuthHeaders(token: string) → object` — constructs the `Authorization: Bearer <token>` header object used by authenticated dashboard requests.

# 21. Module Dependency Map

Arrows indicate import / dependency direction. Leaf modules have no internal dependencies.

## Backend

```
main.py
├── app.api.v1.router
│   ├── endpoints/auth.py        → core/security, repositories/instructor_repository
│   ├── endpoints/analysis.py    → analysis/fingerprint, analysis/scorer
│   ├── endpoints/instructor.py  → models/*, services/encryption_service
│   ├── endpoints/student.py     → repositories/*, services/sanitization_service
│   ├── endpoints/repository.py  → models/repository
│   └── endpoints/health.py      → services/storage_service
├── app.core
│   ├── config.py                (leaf — reads .env)
│   ├── database.py              → config, models/base
│   └── security.py              → config
├── app.models                   → models/base  (all models are leaves)
├── app.repositories             → models/*, schemas/*
├── app.services
│   ├── analysis_service.py      → analysis/*
│   ├── archival_service.py      → models/*
│   ├── encryption_service.py    → core/config
│   ├── sanitization_service.py  (leaf)
│   └── storage_service.py       → core/config
└── app.analysis
    ├── tokenizer.py             (leaf)
    ├── fingerprint.py           (leaf)
    └── scorer.py                (leaf)
```

## Frontend

```
main.tsx
└── App.tsx
    ├── contexts/AuthContext.tsx      → types/auth
    ├── pages/InstructorLogin         → api/authApi, contexts/AuthContext
    ├── pages/InstructorRegister      → api/authApi, contexts/AuthContext
    ├── pages/InstructorProfile       → api/authApi, contexts/AuthContext
    ├── pages/Dashboard               → components/dashboard/*, hooks/useDashboardData
    ├── pages/AssignmentDetails       → components/assignment-details/*,
hooks/useAssignmentDetailsData
    ├── pages/ComparisonView          → components/comparison/*, hooks (inline)
    ├── pages/Repositories            → components/ui/*, lib/apiClient
    └── pages/StudentPortal           → components/student-portal/*,
hooks/useStudentPortalSubmission

Shared utilities across all pages:
    lib/apiClient.ts                  (base Axios instance used everywhere)
    lib/generateComparisonPDF.ts      → components/comparison/types
    lib/generatePDFReport.ts          → components/assignment-details/utils
    components/ui/*                   (Toast, Spinner, ConfirmModal, ErrorBoundary)
```

*End of Document*